

Plotting in R

Author: [Mohammad Khoshroo](#)

From Base Graphics to ggplot2

In this tutorial, we explore two main approaches to plotting in R and use real-world data to examine different statistical distributions.

Part 1: Base R Graphics

Base R Graphics is a simple and fast way to create basic plots without the need to install additional packages.

data()

- `data()` loads built-in datasets (or lists available ones)
- Without arguments (`data()`), it shows available datasets
- With an argument (`data(example_dataset)`), it loads the dataset `example_dataset` from the **datasets** package into your environment
- After this, you can directly use `example_dataset`
- Useful commands:

```
head(example_dataset)      # first rows
str(example_dataset)       # structure of the dataset
summary(example_dataset)   # statistical summary
?example_dataset           # help/documentation
...

### `plot(...)`

**general template** for calling the `plot()` function
```r
plot(x, y,
 type = "p", # "p"=points, "l"=Lines, "b"=both, "h"=vertical lines
 main = "Title", # main title of the plot
 sub = "Subtitle", # subtitle under the plot
 xlab = "X-axis", # label for x-axis
 ylab = "Y-axis", # label for y-axis
 xlim = c(min, max), # range for x-axis
 ylim = c(min, max), # range for y-axis
 pch = 1, # plotting character/symbol (1=open circle, 19=solid circle, etc.)
 col = "black", # color of points/lines
 cex = 1, # size of points (default = 1)
 lty = 1, # line type (1=solid, 2=dashed, etc.)
 lwd = 1, # line width
 axes = TRUE, # whether to draw axes
 frame.plot = TRUE # whether to draw a box around the plot
)
```

LANGUAGE-R

### Examples

#### Scatter Plot

When you give two numeric vectors ( `x` and `y` ), it creates a **scatter plot**

```
for background
par(bg = "white")

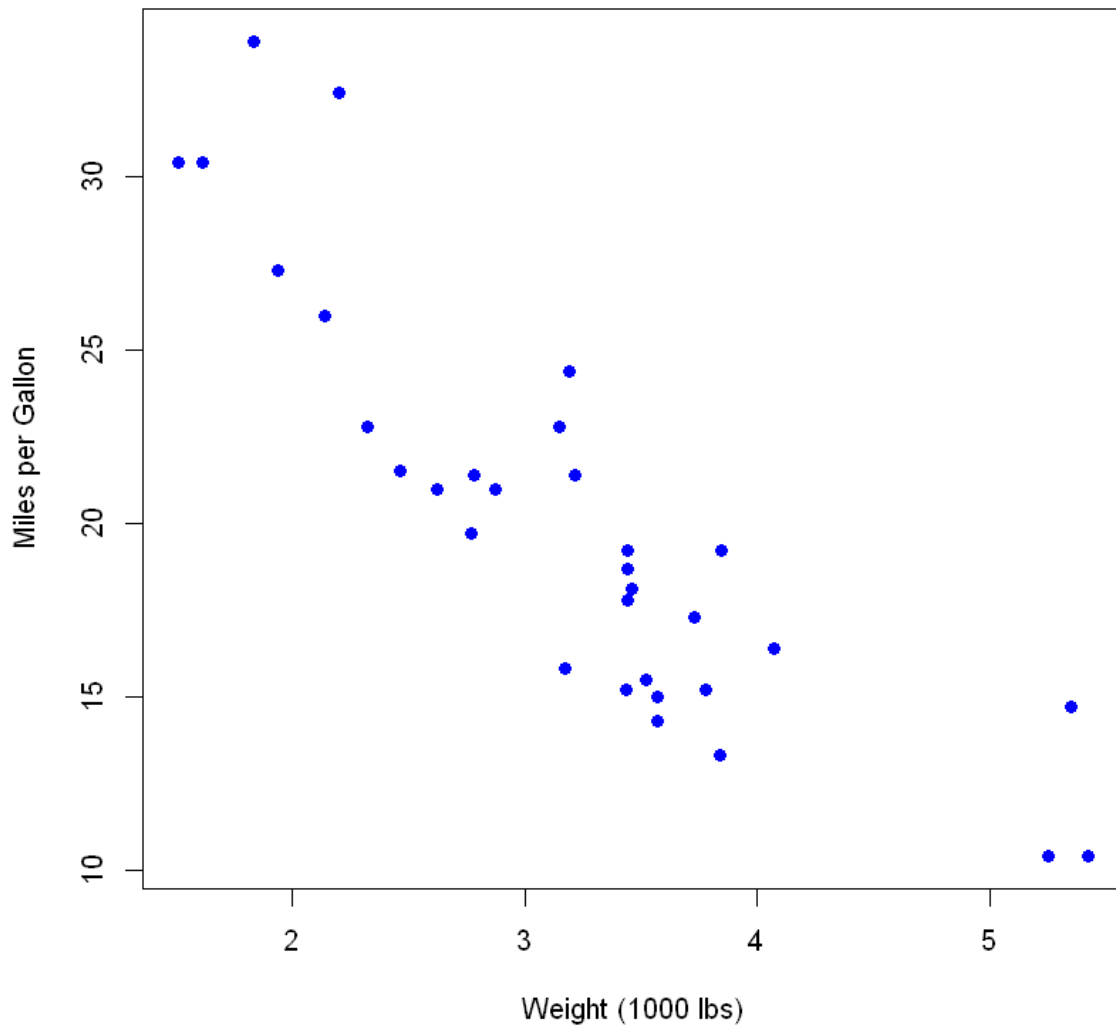
Load the mtcars dataset (car data)
data(mtcars)

Scatter plot of weight vs. fuel consumption
```

LANGUAGE-R

```
plot(mtcars$wt, mtcars$mpg,
 main = "Relationship Between Car Weight and Fuel Consumption",
 xlab = "Weight (1000 lbs)",
 ylab = "Miles per Gallon",
 pch = 19,
 col = "blue")
```

Relationship Between Car Weight and Fuel Consumption



## Line Plot

R chooses a **line plot automatically** for objects that have a natural order, e.g. :

- **Time series ( ts ) objects** → `plot.ts()` draws a line plot by default.
- **Some model objects** (like `lm` ) → default plots often include lines.

```
for background
par(bg = "white")

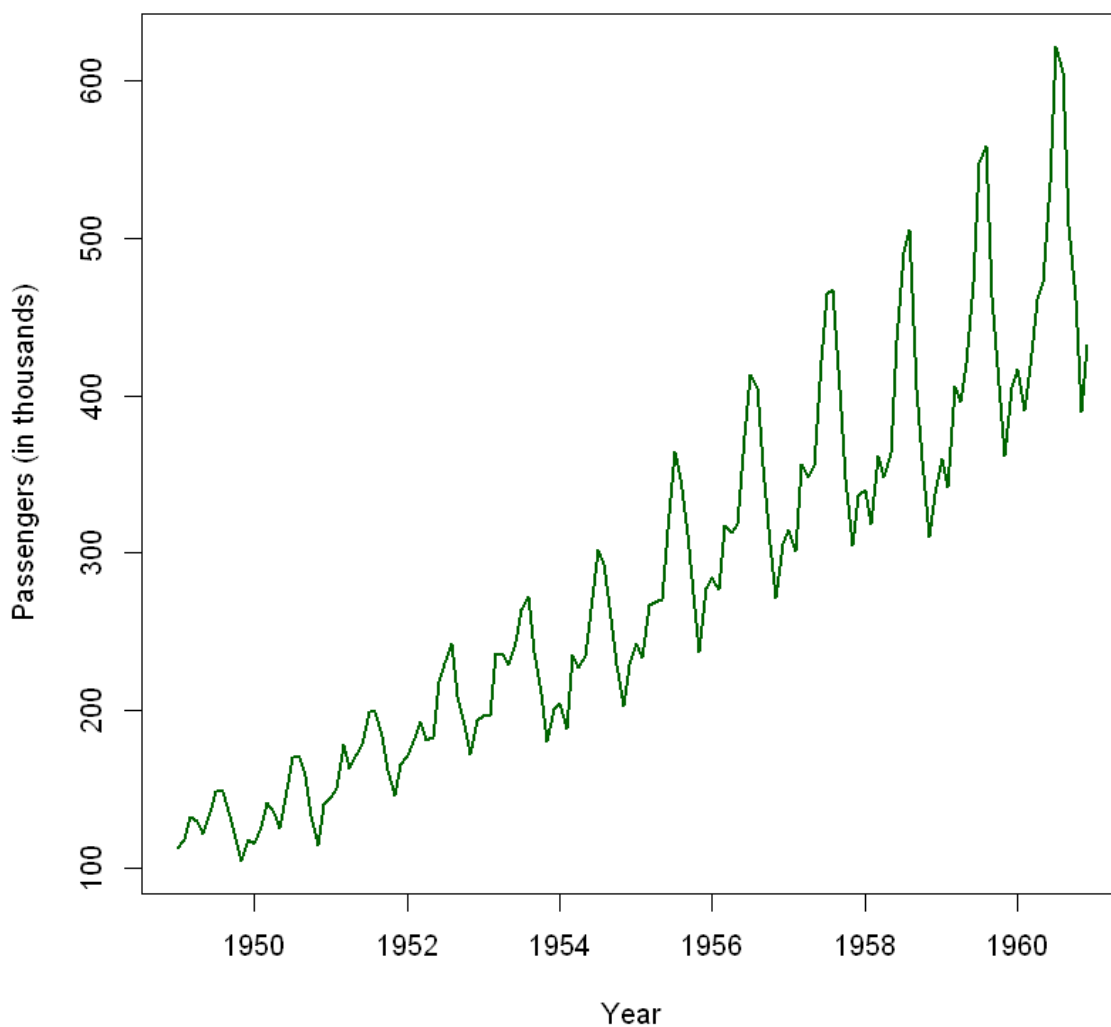
AirPassengers dataset (airline passengers 1949-1960)
data(AirPassengers)

structure of the dataset
str(AirPassengers)

plot(AirPassengers,
 main = "Number of Airline Passengers Over Time",
 xlab = "Year",
 ylab = "Passengers (in thousands)",
 col = "darkgreen",
 lwd = 2)
```

LANGUAGE-R

### Number of Airline Passengers Over Time



## How `plot()` Chooses What to Draw !?

In R, the function `plot()` is **generic**. This means that the type of plot it produces depends on the *class* of the object you give it.

When you call:

```
plot(x)
```

LANGUAGE-R

R first checks the class of `x` with `class(x)`. Then it looks for a method named `plot.<classname>`.

- If such a method exists, R calls it.
- If no class-specific method is found, it falls back to `plot.default()`.

This mechanism explains why the same `plot()` function can behave differently for different datasets.

## Examples

### 1. Numeric vector

```
x <- 1:10
plot(x)
```

LANGUAGE-R

- `class(x) = "integer" → handled by plot.default()`
- Result: a scatter plot of values versus their index

### 2. Two numeric vectors

```
plot(1:10, rnorm(10))
```

LANGUAGE-R

- Both are numeric → `plot.default()`
- Result: a scatter plot of `y` against `x`

### 3. Time series ( `ts` object)

```
data(AirPassengers)
plot(AirPassengers)
```

LANGUAGE-R

- `class(AirPassengers) = "ts" → handled by plot.ts()`
- Result: a time series line chart with years on the x-axis

### 4. Factor

```
f <- factor(c("A", "B", "A", "C", "B", "A"))
plot(f)
```

LANGUAGE-R

- `class(f) = "factor" → handled by plot.factor()`
- Result: a bar plot showing the frequency of each category

### 5. Data frame

```
plot(iris)
```

LANGUAGE-R

- `class(iris) = "data.frame" → handled by plot.data.frame()`
- Result: a scatterplot matrix of all numeric columns.



## In short

The `plot()` function in R is not tied to a single visualization style. Instead, it dispatches to a class-specific method such as:

- `plot.default()` → numeric vectors, scatter plots
- `plot.ts()` → time series objects, line plots
- `plot.factor()` → factors, bar plots
- `plot.data.frame()` → data frames, scatterplot matrices

This design makes `plot()` flexible and context-aware: you can call the same function on many different data types, and R will choose the most appropriate plot automatically.

### `par()`

- `par()` is a function that sets global graphical parameters in R
- The argument `bg` specifies the background color of the plotting device (the drawing window)
- `par(bg = "white")` means the background is set to white

💡 **Note:** `par()` affects all subsequent plots. If you want to reset parameters, you can save them before:

```
old.par <- par()
par(bg = "white")
par(old.par) # restores old settings
```

LANGUAGE-R

## hist() (Histogram)

general template for calling the `hist()` function

```
hist(x,
 breaks = "Sturges", # method or number of bins (e.g., "Sturges", "FD", or a number)
 freq = NULL, # TRUE = show counts, FALSE = show densities
 probability = !freq, # TRUE = plot density instead of counts
 main = "Histogram Title", # main title
 sub = NULL, # subtitle
 xlab = "X-axis Label", # label for x-axis
 ylab = "Y-axis Label", # label for y-axis
 col = "gray", # fill color of bars
 border = "black", # color of bar borders
 xlim = NULL, # x-axis limits
 ylim = NULL, # y-axis limits
 labels = FALSE, # add frequency labels on bars
 include.lowest = TRUE, # include lowest value in first bin
 right = TRUE, # if TRUE, bins are right-closed
 density = NULL, # shading lines density
 angle = 45 # angle of shading lines (if density used)
)
```

`x` must be a **numeric vector**.

When you give a function like `plot()` or `hist()` just a **numeric vector** `x`, R automatically decides **how to display it**.

- **x-axis** → the numeric range of the data (bins)
- **y-axis** → the count of values in each bin (frequency)
- **Default display** → histogram (bar plot)

Essentially: R counts how many values fall into each interval and draws a bar for each bin.

### 💡 Note:

- `breaks` controls **number and position of bins**.
- `freq = TRUE` → y-axis = counts, `freq = FALSE` → y-axis = density.

Example:

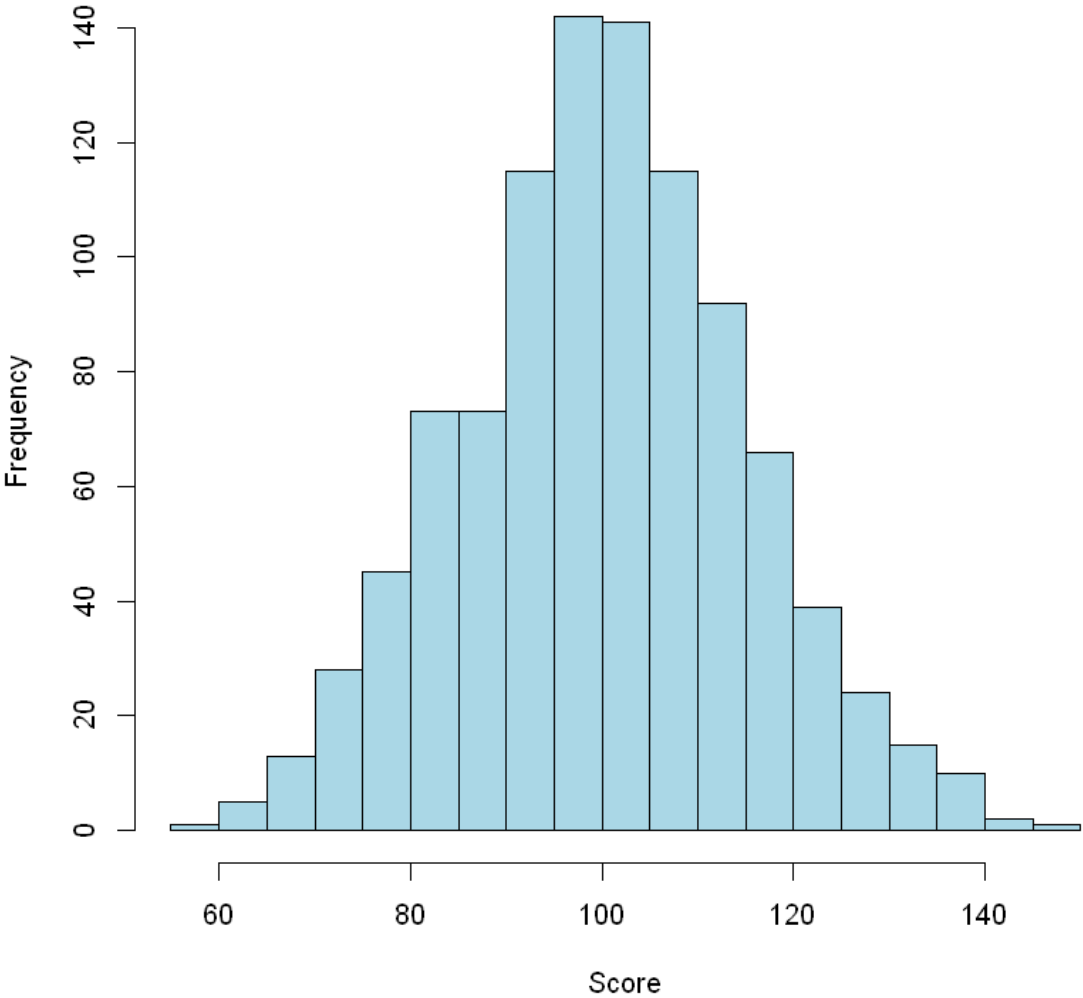
```
for background
par(bg = "white")

Generate normally distributed data
set.seed(123)
normal_data <- rnorm(1000, mean = 100, sd = 15)

structure of the dataset
str(normal_data)

Plot histogram
hist(normal_data,
 main = "Normal Distribution (IQ Score)",
 xlab = "Score",
 ylab = "Frequency",
 col = "lightblue",
 breaks = 30)
```

Normal Distribution (IQ Score)



## curve(add = FALSE)

general template for calling the `curve()` function

```
curve(expr,
 from = NULL, # starting x value (required)
 to = NULL, # ending x value (required)
 n = 101, # number of points to plot (smoothness)
 add = FALSE, # if TRUE, add to existing plot; if FALSE, create new plot
 type = "l", # plot type ("l" = line, "p" = points, etc.)
 col = "blue", # color of the curve
 lwd = 2, # line width
 lty = 1, # line type (1 = solid, 2 = dashed, etc.)
 xlab = "x", # label for x-axis
 ylab = "y", # label for y-axis
 main = "Function Curve", # main title
 xlim = NULL, # limits for x-axis
 ylim = NULL, # limits for y-axis
 log = "", # log scale ("x", "y", or "xy")
 axes = TRUE, # whether to draw axes
 frame.plot = TRUE # whether to draw a box around the plot
)
```

LANGUAGE-R

`expr` is a **mathematical expression** (Example:  $x^2$ ) or a **function** that uses `x` as the variable.

R internally creates `x` values from `from` to `to`, evaluates `y = expr`, and plots the resulting curve.

It **creates a new plot** with axes, labels, and scales.

R automatically determines the range of `y` based on computed values.

`curve()` generates a smooth line for a mathematical expression or function over a given x-range.

### 💡 Note:

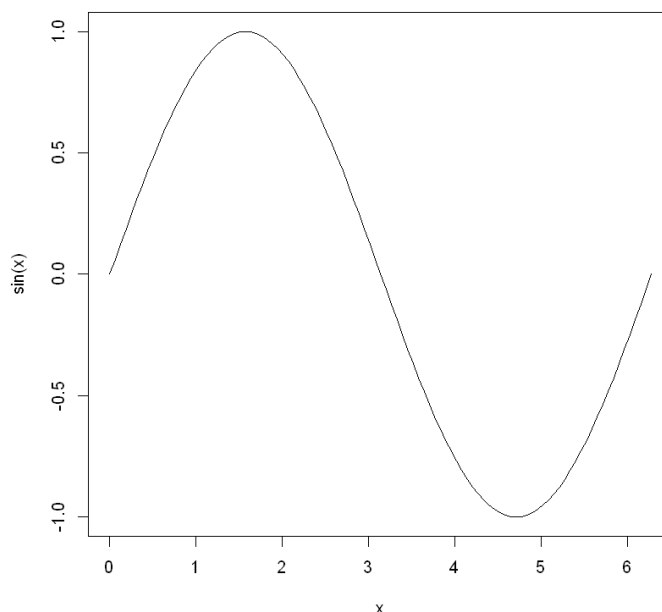
- `n` controls smoothness — more points → smoother curve.
- You can use **any valid R function**: `curve(exp(x))`, `curve(log(x))`, etc.
- Works great for **visualizing mathematical functions** or **adding fitted lines**.

Example:

```
for background
par(bg = "white")

curve(sin(x), from = 0, to = 2*pi)
```

LANGUAGE-R



## boxplot() (Box-and-Whisker Plot)

general template for calling the `boxplot()` function

```
boxplot(x,
 data = NULL, # optional data frame (used when specifying formula)
 formula = NULL, # formula interface: y ~ group
 main = "Boxplot Title", # main title
 sub = NULL, # subtitle
 xlab = "X-axis Label", # label for x-axis
 ylab = "Y-axis Label", # label for y-axis
 col = "lightblue", # fill color of boxes
 border = "black", # color of box borders
 notch = FALSE, # if TRUE, draws a notch (confidence interval around median)
 outline = TRUE, # show outlier points
 names = NULL, # names for each group
 horizontal = FALSE, # if TRUE, draw boxes horizontally
 ylim = NULL, # limits for y-axis
 xlim = NULL, # limits for x-axis
 las = 1, # orientation of axis labels (0=parallel, 1=horizontal)
 at = NULL, # positions of boxes (numeric vector)
 width = NULL, # widths of boxes
 add = FALSE, # if TRUE, add to an existing plot
 pars = NULL # list of graphical parameters
)
```

LANGUAGE-R

`x` can be:

- A **numeric vector** → creates one boxplot.
- A **list** or **data frame** of numeric vectors → creates multiple boxplots (side by side).
- A **formula** (e.g. `y ~ group`, with `data=`) → creates grouped boxplots based on categories.

## What the boxplot shows !?

- **Box** → interquartile range (IQR: 25th–75th percentile)
- **Line inside box** → median (50th percentile)
- **Whiskers** → extend to the smallest/largest values within  $1.5 \times \text{IQR}$
- **Dots beyond whiskers** → outliers

### 💡 Notes:

- Use `horizontal = TRUE` to flip the plot.
- Use `notch = TRUE` to show a **notch** around the median (rough CI).
- Use `outline = FALSE` to hide outlier points.
- When using `formula`, R automatically groups data by the right-hand variable:

```
boxplot(mpg ~ cyl, data = mtcars)
```

LANGUAGE-R

→ creates separate boxplots of `mpg` for each `cyl` (number of cylinders).

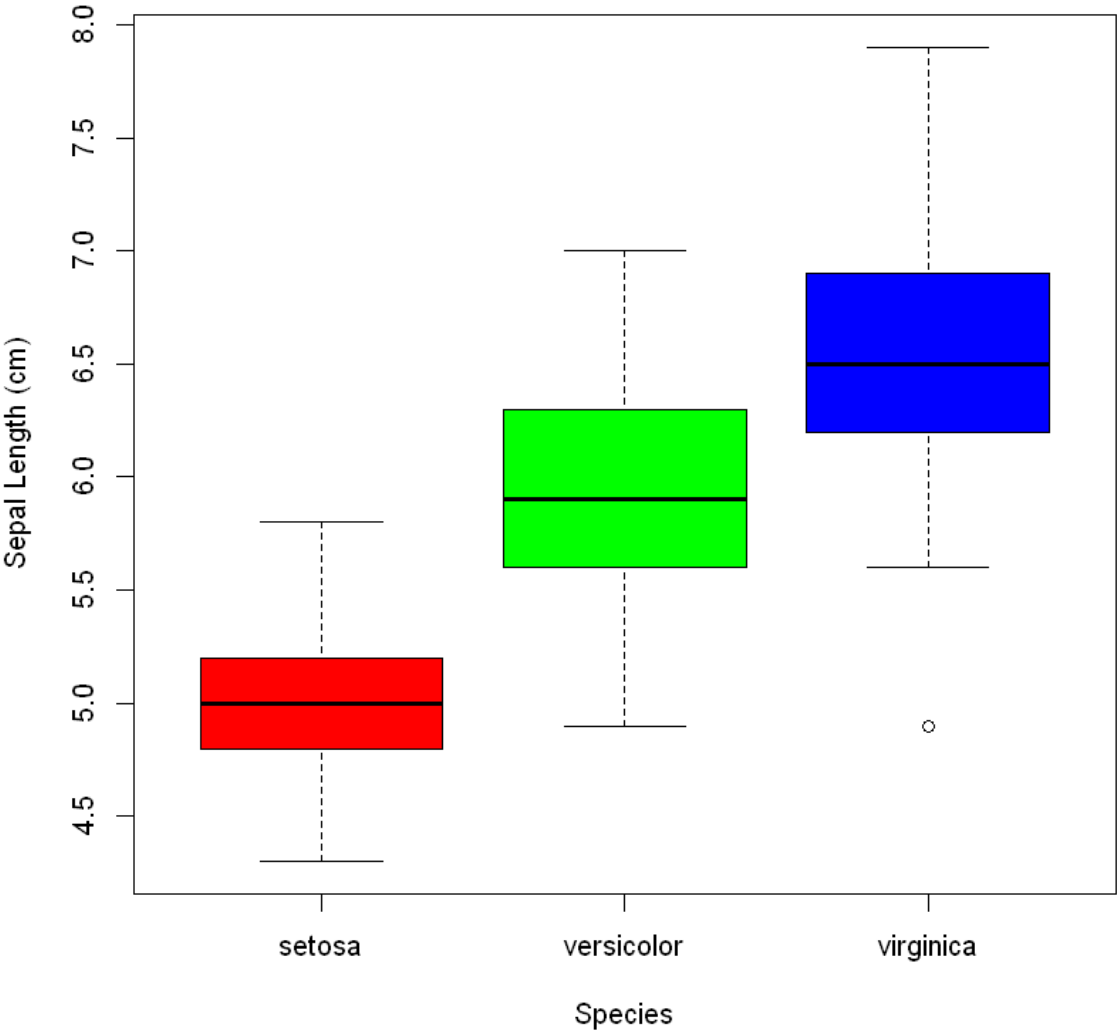
```
for background
par(bg = "white")

iris dataset (measurements of iris flowers)
data(iris)

boxplot(Sepal.Length ~ Species, data = iris,
 main = "Comparison of Sepal Length Across Three Iris Species",
 xlab = "Species",
 ylab = "Sepal Length (cm)",
 col = c("red", "green", "blue"))
```

LANGUAGE-R

Comparison of Sepal Length Across Three Iris Species



## pie() (Pie Chart)

general template for calling the `pie()` function

```
pie(x,
 labels = names(x), # labels for each slice
 main = "Pie Chart Title", # main title
 col = NULL, # colors for slices (can use a vector or palette)
 clockwise = FALSE, # draw slices clockwise (default = counterclockwise)
 radius = 1, # radius of the pie (default = 1)
 init.angle = 0, # starting angle (in degrees)
 border = NULL, # color of slice borders
 density = NULL, # density of shading lines
 angle = 45, # angle for shading lines
 edges = 200, # smoothness of the circle (more edges = smoother)
 cex = 1, # text size for labels
 font = 1, # text font
 col.main = "black", # color of main title
 frame.plot = FALSE # whether to draw a box around the plot
)
```

LANGUAGE-R

- `x` must be a **numeric vector of positive values**
- Each value in `x` represents the **size of a slice** (R calculates each slice's angle based on its proportion of the total)
- Labels are taken from `names(x)` unless specified manually
- R converts values to proportions:

$$\text{Angle}_i = 360^\circ \times \frac{x_i}{\sum x}$$

- Then, it draws a **circle** divided into slices corresponding to these proportions
- Use `clockwise = TRUE` to reverse the direction of slice drawing
- You can adjust `init.angle` to rotate where the first slice starts (e.g., `init.angle = 90` starts at the top)

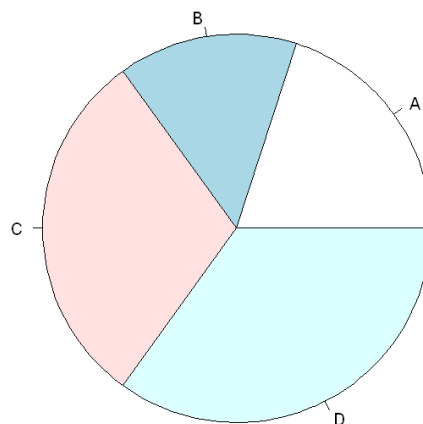
Example:

```
for background
par(bg = "white")

x <- c(20, 15, 30, 35)
labels <- c("A", "B", "C", "D")
pie(x, labels = labels, main = "Category Distribution")
```

LANGUAGE-R

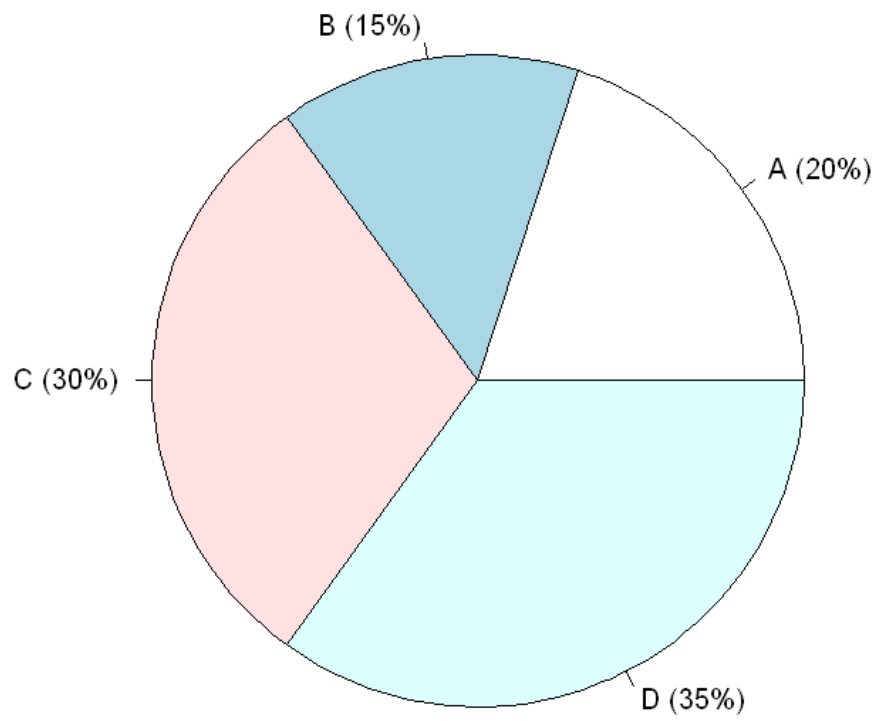
Category Distribution



To show percentages:

```
pie(x, labels = paste0(labels, " (", round(100 * x/sum(x)), "%)"))
```

LANGUAGE-R





## image() (Heatmap)

general template for calling the `image()` function

```
image(x,
 y,
 z,
 main = "Image Plot", # main title
 sub = NULL, # subtitle
 xlab = "X-axis", # label for x-axis
 ylab = "Y-axis", # label for y-axis
 col = heat.colors(12), # color palette (e.g., heat.colors, topo.colors, terrain.colors)
 xlim = range(x), # limits for x-axis
 ylim = range(y), # limits for y-axis
 zlim = range(z, na.rm = TRUE), # limits for color mapping (z-values)
 axes = TRUE, # whether to draw axes
 asp = NA, # aspect ratio (1 = equal scaling)
 add = FALSE, # add to an existing plot (default = new plot)
 useRaster = TRUE # faster rendering for large matrices
)
```

LANGUAGE-R

`image()` visualizes **2D numeric data (matrices)** as a grid of colored rectangles — each color corresponds to a numeric value, making it ideal for **heatmaps** or **surface-like plots**.

- `image()` displays **matrix or grid data** as a colored grid (like a heatmap)
- Each element of `z` is shown as a **colored rectangle** whose color corresponds to its numeric value

**Arguments meaning:**

- `x` → vector of x coordinates (or row indices if omitted)
- `y` → vector of y coordinates (or column indices if omitted)
- `z` → **matrix** of numeric values (required)

If `x` and `y` are omitted, R assumes they are the **row and column indices** of `z`

Example:

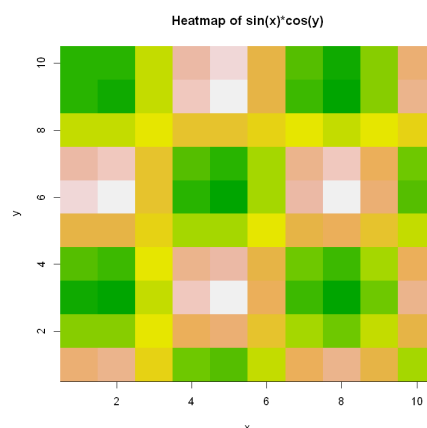
This creates a colored grid where color intensity represents the value of `sin(x)*cos(y)` at each grid point.

```
for background
par(bg = "white")

Generate data
x <- 1:10
y <- 1:10
z <- outer(x, y, function(a, b) sin(a) * cos(b))

Image plot
image(x, y, z,
 main = "Heatmap of sin(x)*cos(y)",
 col = terrain.colors(20))
```

LANGUAGE-R



# Common Plot Helper Functions

## ● In short:

Functions that *create* plots (like `plot()`, `hist()`, `barplot()`) start fresh and set up the plotting area.

Functions that *add* elements (like `points()`, `lines()`, `text()`, `legend()`) need an existing plot.

here's a list of **helper-adder functions** you can use after plotter functions ex. `plot()` in:

- `grid()` → adds a background grid to the plot
- `abline()` → adds straight lines (horizontal, vertical, or regression lines)
- `lines()` → adds lines connecting points to an existing plot
- `points()` → adds extra points to an existing plot
- `text()` → adds text labels at specific coordinates
- `mtext()` → adds text in the plot margins (e.g., side titles)
- `title()` → adds or changes the main title, axis labels, or subtitle
- `legend()` → adds a legend to the plot
- `arrows()` → draws arrows between points
- `segments()` → draws line segments between pairs of points
- `rect()` → draws rectangles
- `polygon()` → draws a filled polygon
- `curve(add = TRUE)` → draws mathematic curves and functions

These functions are usually called **after** `plot()` to enhance or annotate the graphic.

### `grid()`

```
grid(nx = NULL, ny = NULL,
 col = "gray", # color of grid lines
 lty = 2, # line type (2 = dashed)
 lwd = 1 # line width
)
```

LANGUAGE-R

### `abline()`

```
abline(h = NULL, v = NULL, # horizontal or vertical lines
 a = NULL, b = NULL, # intercept (a) and slope (b) of line
 reg = NULL, # regression model object, e.g. lm()
 col = "black", # line color
 lty = 1, # line type
 lwd = 1 # line width
)
```

LANGUAGE-R

### `lines()`

```
lines(x, y,
 col = "black", # line color
 lty = 1, # line type
 lwd = 1, # line width
 type = "l" # usually "l" for line
)
```

LANGUAGE-R

### `points()`

```
points(x, y,
 pch = 1, # symbol type
 col = "black", # point color
 cex = 1 # point size
)
```

LANGUAGE-R

## text()

LANGUAGE-R

```
text(x, y,
 labels, # text to display
 pos = NULL, # 1=below, 2=left, 3=above, 4=right of (x,y)
 col = "black", # text color
 cex = 1 # text size
)
```

## mtext()

LANGUAGE-R

```
mtext(text,
 side = 3, # 1=bottom, 2=left, 3=top, 4=right
 line = 0, # margin line to place text
 col = "black", # text color
 cex = 1 # text size
)
```

## title()

LANGUAGE-R

```
title(main = NULL, # main title
 sub = NULL, # subtitle
 xlab = NULL, # x-axis label
 ylab = NULL, # y-axis label
 col.main = NULL, # main title color
 col.sub = NULL # subtitle color
)
```

## legend()

LANGUAGE-R

```
legend(x, y, legend,
 col = NULL, # colors for symbols/lines
 pch = NULL, # point symbols
 lty = NULL, # line types
 lwd = NULL, # line widths
 bty = "o", # box type around legend ("o"=box, "n"=none)
 cex = 1 # text size
)
```

## arrows()

LANGUAGE-R

```
arrows(x0, y0, x1, y1,
 col = "black", # arrow color
 lty = 1, # line type
 lwd = 1, # line width
 length = 0.1 # size of arrowhead
)
```

## segments()

```
segments(x0, y0, x1, y1,
 col = "black", # color
 lty = 1, # line type
 lwd = 1 # line width
)
```

LANGUAGE-R

## rect()

```
rect(xleft, ybottom, xright, ytop,
 col = NA, # fill color
 border = "black" # border color
)
```

LANGUAGE-R

## polygon()

```
polygon(x, y,
 col = "gray", # fill color
 border = "black", # border color
 lty = 1, # border line type
 lwd = 1 # border line width
)
```

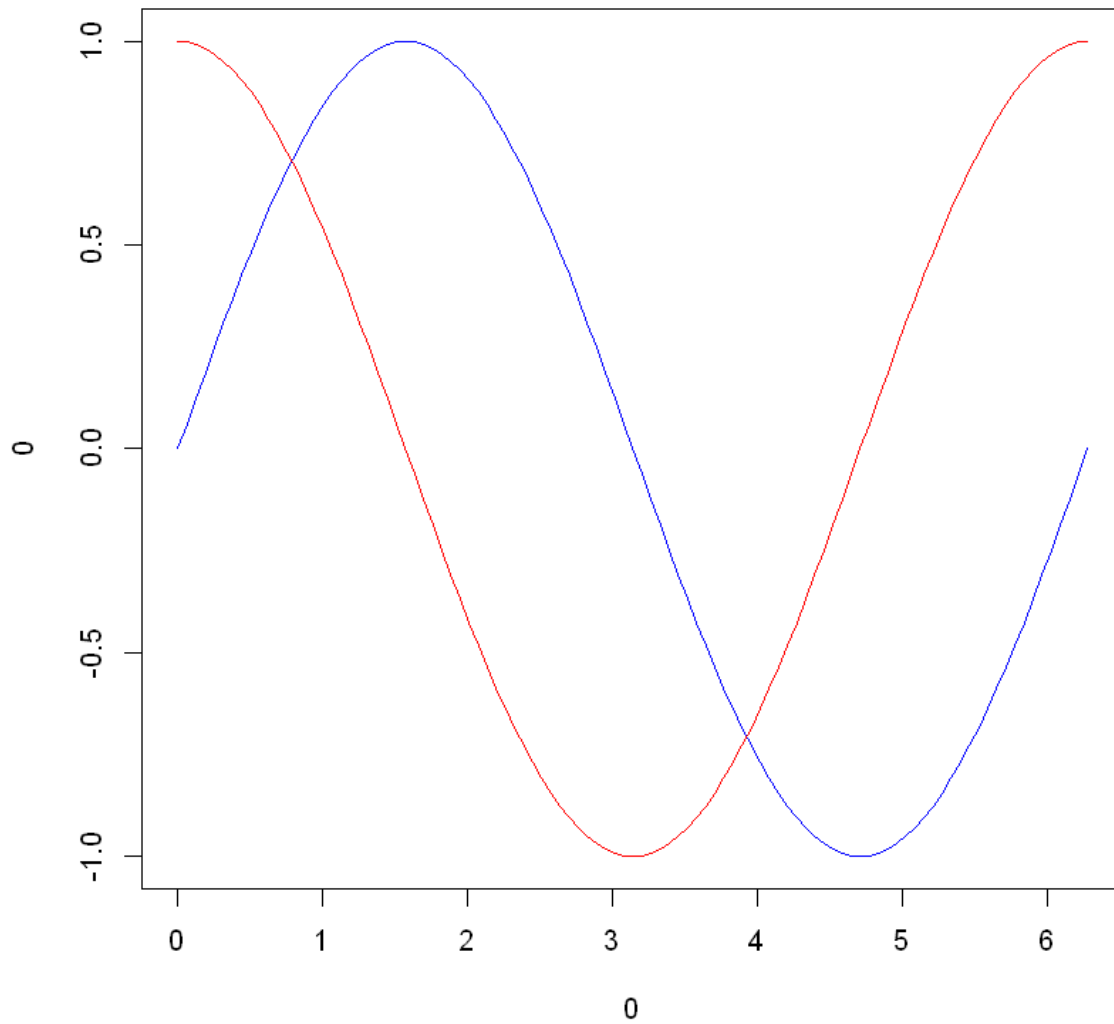
LANGUAGE-R

`curve(add = TRUE)`

Example:

```
plot(0, 0, xlim = c(0, 2*pi), ylim = c(-1, 1), type = "n")
curve(sin(x), from = 0, to = 2*pi, add = TRUE, col = "blue")
curve(cos(x), from = 0, to = 2*pi, add = TRUE, col = "red")
```

LANGUAGE-R



Example:

LANGUAGE-R

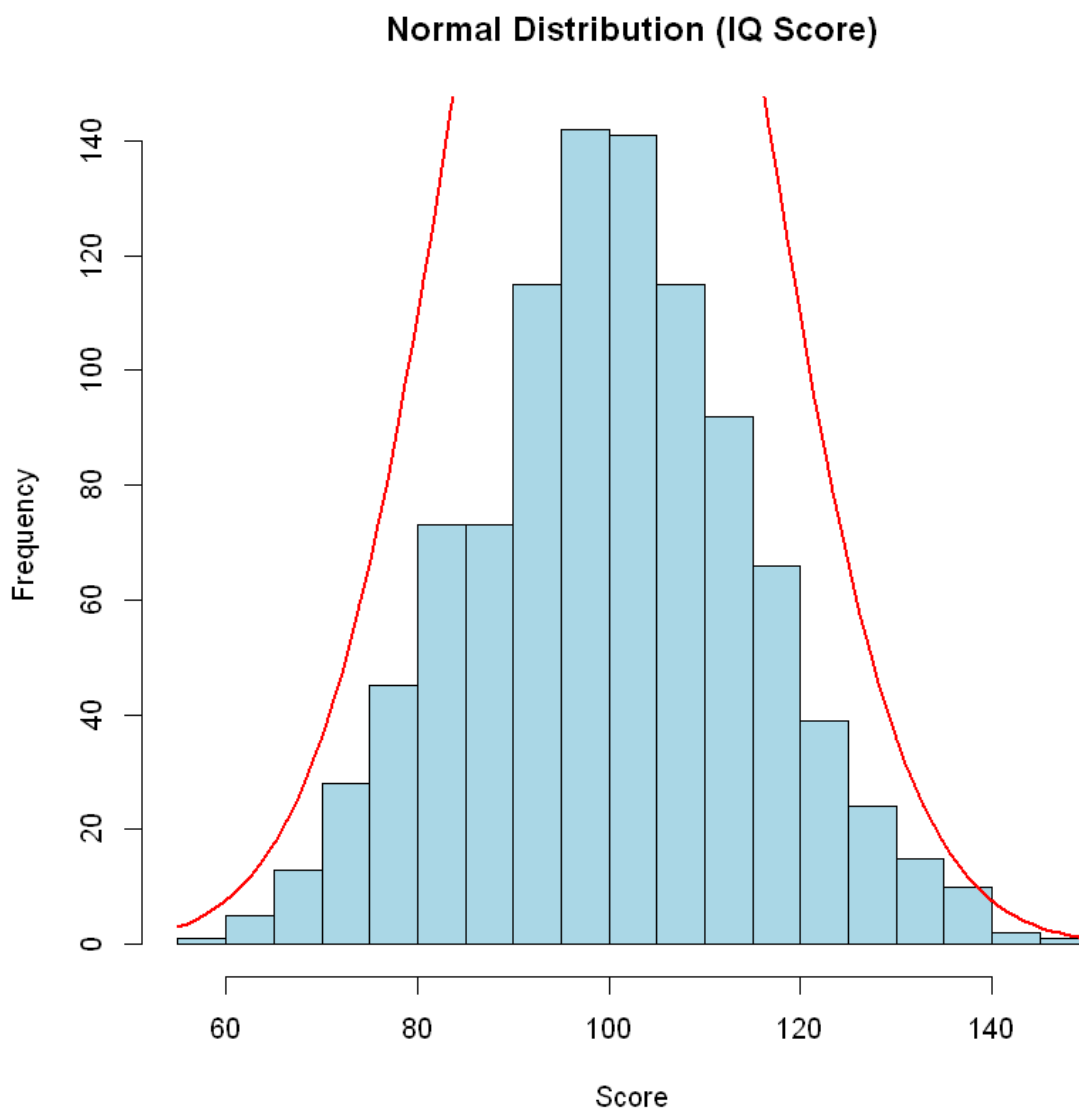
```
for background
par(bg = "white")

Generate normally distributed data
set.seed(123)
normal_data <- rnorm(1000, mean = 100, sd = 15)

structure of the dataset
str(normal_data)

Plot histogram
hist(normal_data,
 main = "Normal Distribution (IQ Score)",
 xlab = "Score",
 ylab = "Frequency",
 col = "lightblue",
 breaks = 30)

Add normal distribution curve
curve(dnorm(x, mean = 100, sd = 15) * 1000 * diff(hist(normal_data, plot = FALSE)$breaks)[1],
 add = TRUE, col = "red", lwd = 2)
```



## Part 2: ggplot() (Grammar of Graphics Plot — from **ggplot2** package)

### Installation & Load

```
Install ggplot2 (if not already installed)
install.packages("ggplot2")

Load the ggplot2 package
library(ggplot2)
```

LANGUAGE-R

### ggplot2 Base Structure

```
ggplot(data = dataset, aes(x = x_variable, y = y_variable)) +
 geom_chart_type()
```

LANGUAGE-R

general template for calling the `ggplot()` function

```
ggplot(data = <DATA>,
 mapping = aes(x = <X>, y = <Y>)) +
 <GEOM_FUNCTION>{
 stat = "identity", # statistical transformation (e.g., "identity", "smooth", "count")
 position = "identity", # how elements are placed (e.g., "stack", "dodge")
 color = "black", # color of lines or points
 fill = "steelblue", # fill color (for bars, areas, etc.)
 size = 1, # line or point size
 alpha = 1 # transparency (0 = transparent, 1 = opaque)
 } +
 labs(
 title = "Plot Title", # main title
 subtitle = "Subtitle",
 x = "X-axis Label",
 y = "Y-axis Label",
 caption = "Source or note"
) +
 theme_minimal() # choose a theme (e.g., classic, light, dark, minimal)
```

LANGUAGE-R

`ggplot()` is the foundation of the **ggplot2** system, based on the “Grammar of Graphics”  
You **build a plot step-by-step** by adding layers with `+`

**Basic components:**

1. `data =` → the dataset (usually a data frame)
2. `aes()` → *aesthetic mappings*, define which columns go to x, y, color, size, etc
3. `geom_...()` → the geometric layer (e.g., points, lines, bars, etc.)

# Common geom layers:

Function	Type	Example	Model / Usage
geom_point()	Scatter Plot	geom_point(aes(x, y))	scatter / relationship model
geom_line()	Line Plot	geom_line(aes(time, value))	trend / time-series
geom_bar()	Bar Chart	geom_bar(aes(category))	categorical counts
geom_histogram()	Histogram	geom_histogram(aes(x), bins=10)	distribution
geom_boxplot()	Boxplot	geom_boxplot(aes(group, value))	summary / spread / outliers
geom_density()	Density Plot	geom_density(aes(x))	density estimation
geom_smooth()	Smoothed Line / Trend	geom_smooth(method="lm")	regression / linear model
geom_area()	Area Plot	geom_area(aes(x, y))	cumulative / trend
geom_violin()	Violin Plot	geom_violin(aes(group, value))	distribution shape
geom_col()	Bar (Pre-aggregated)	geom_col(aes(x, y))	aggregated bar
geom_text()	Text / Labels	geom_text(aes(label))	annotation / label
geom_errorbar()	Error Bars	geom_errorbar(aes(ymin, ymax))	uncertainty / confidence interval
geom_path()	Connected Path	geom_path(aes(x, y))	sequential / ordered data
geom_tile()	Heatmap	geom_tile(aes(x, y, fill))	matrix / intensity visualization
geom_ribbon()	Ribbon / Band	geom_ribbon(aes(ymin, ymax))	interval / uncertainty

- You can add multiple layers:

```
ggplot(mtcars, aes(wt, mpg)) +
 geom_point(color = "blue") +
 geom_smooth(method = "lm", se = FALSE, color = "red")
```

LANGUAGE - R



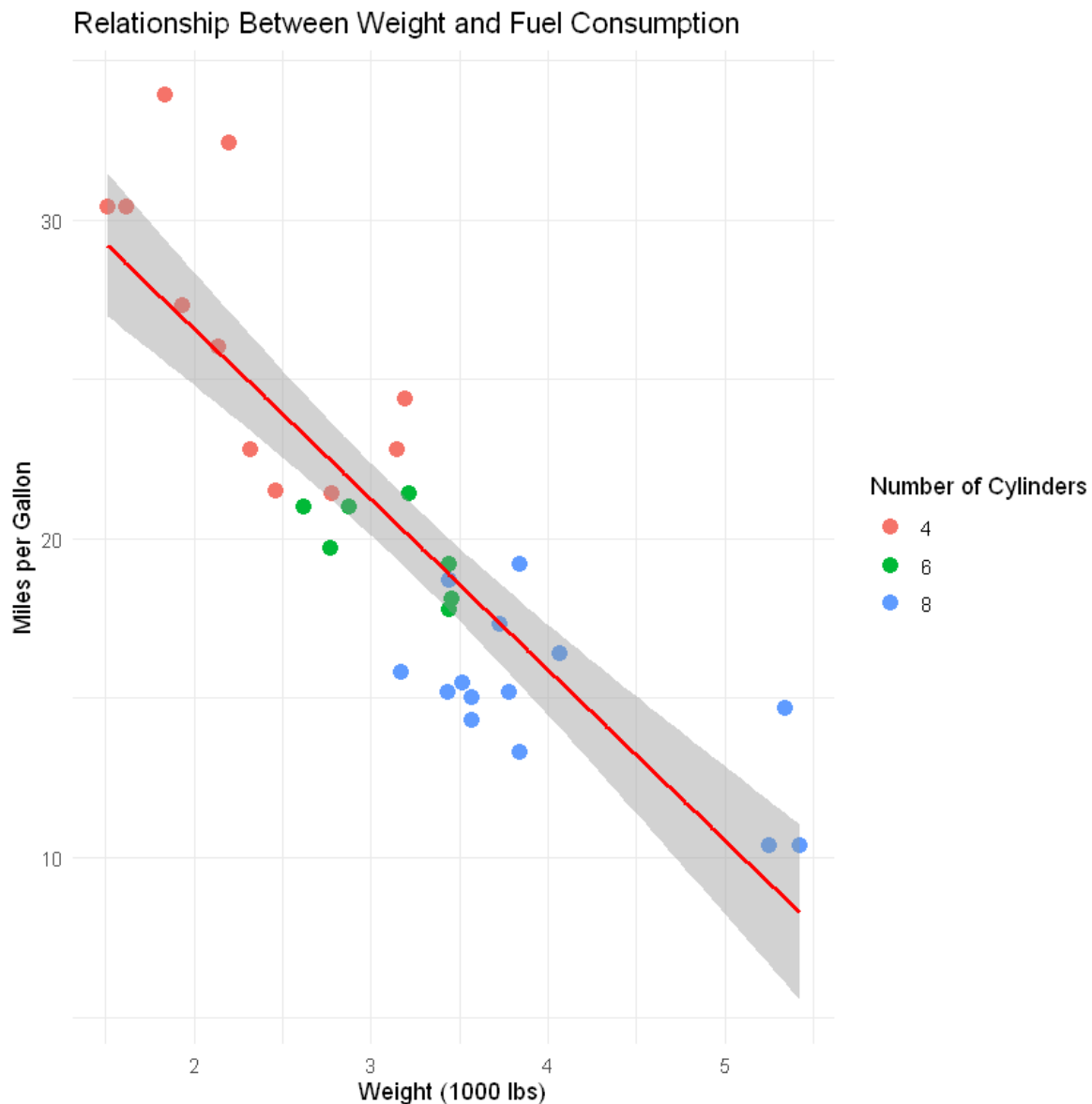
# Plots with ggplot2 (Examples)

## Scatter Plot

```
mtcars dataset
ggplot(mtcars, aes(x = wt, y = mpg)) +
 geom_point(aes(color = factor(cyl)), size = 3) +
 # scatter plot points, color by number of cylinders
 geom_smooth(method = "lm", se = TRUE, color = "red") +
 # regression line (linear model) with confidence interval
 # Explanation:
 # - geom_smooth() draws a smoothed line or trend over the data
 # - method = "lm" specifies a linear model (linear regression)
 # - se = TRUE displays the confidence interval around the regression line
 labs(title = "Relationship Between Weight and Fuel Consumption",
 x = "Weight (1000 lbs)",
 y = "Miles per Gallon",
 color = "Number of Cylinders") +
 # Adds labels and legend title
 theme_minimal()
 # Minimal theme for cleaner appearance
```

LANGUAGE-R

`geom_smooth()` using formula = 'y ~ x'



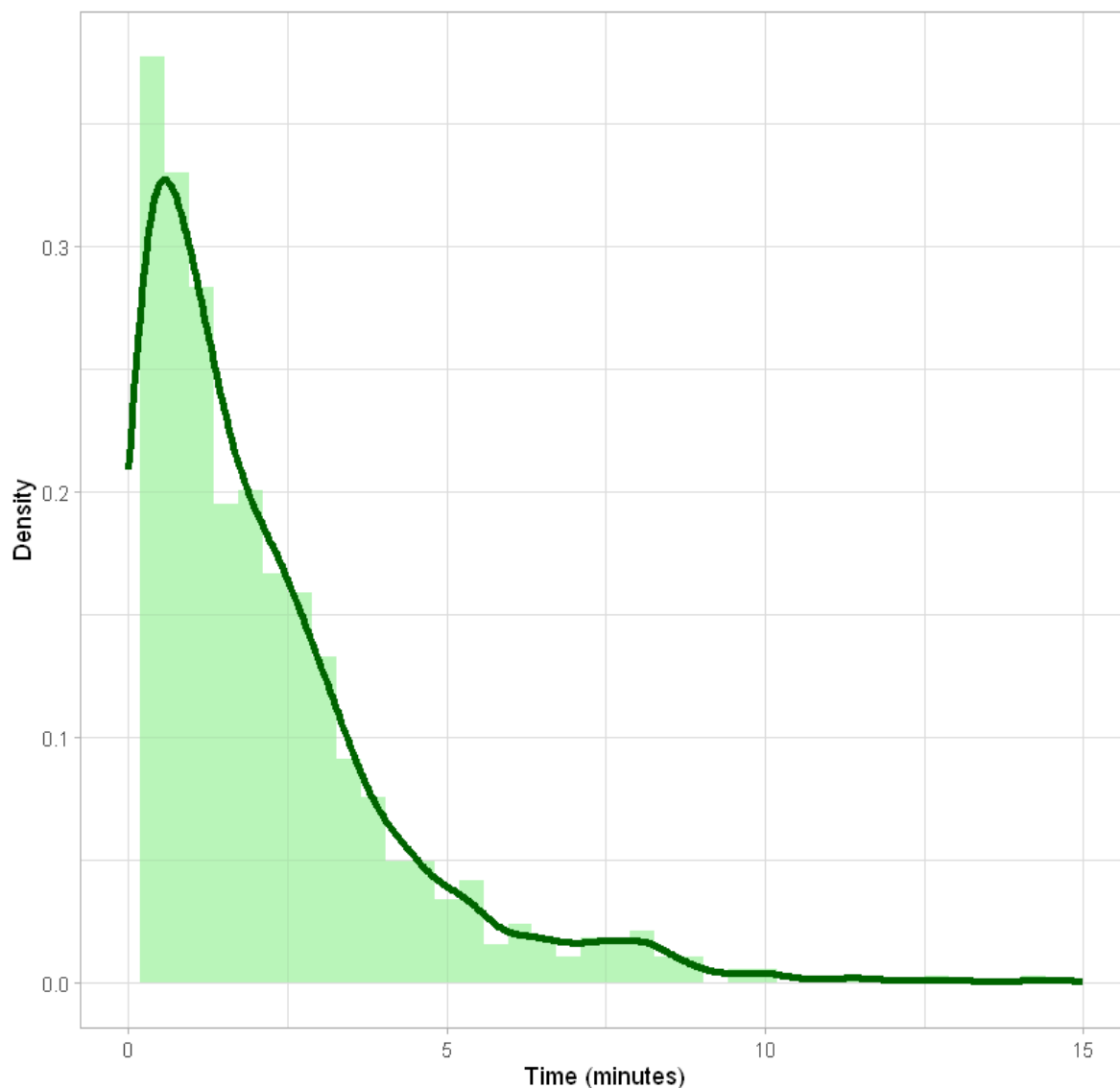
## Density Curve (Exponential Distribution)

LANGUAGE-R

```
Exponential distribution (e.g., time between calls)
set.seed(123)
df_exp <- data.frame(
 wait_time = rexp(1000, rate = 0.5) # Waiting time (minutes)
)

ggplot(df_exp, aes(x = wait_time)) +
 geom_histogram(aes(y = after_stat(density)),
 bins = 40,
 fill = "lightgreen",
 alpha = 0.6) +
 geom_density(color = "darkgreen", linewidth = 1.5) +
 xlim(0, 15) +
 labs(title = "Exponential Distribution - Time Between Calls",
 x = "Time (minutes)",
 y = "Density") +
 theme_light()
```

Exponential Distribution – Time Between Calls

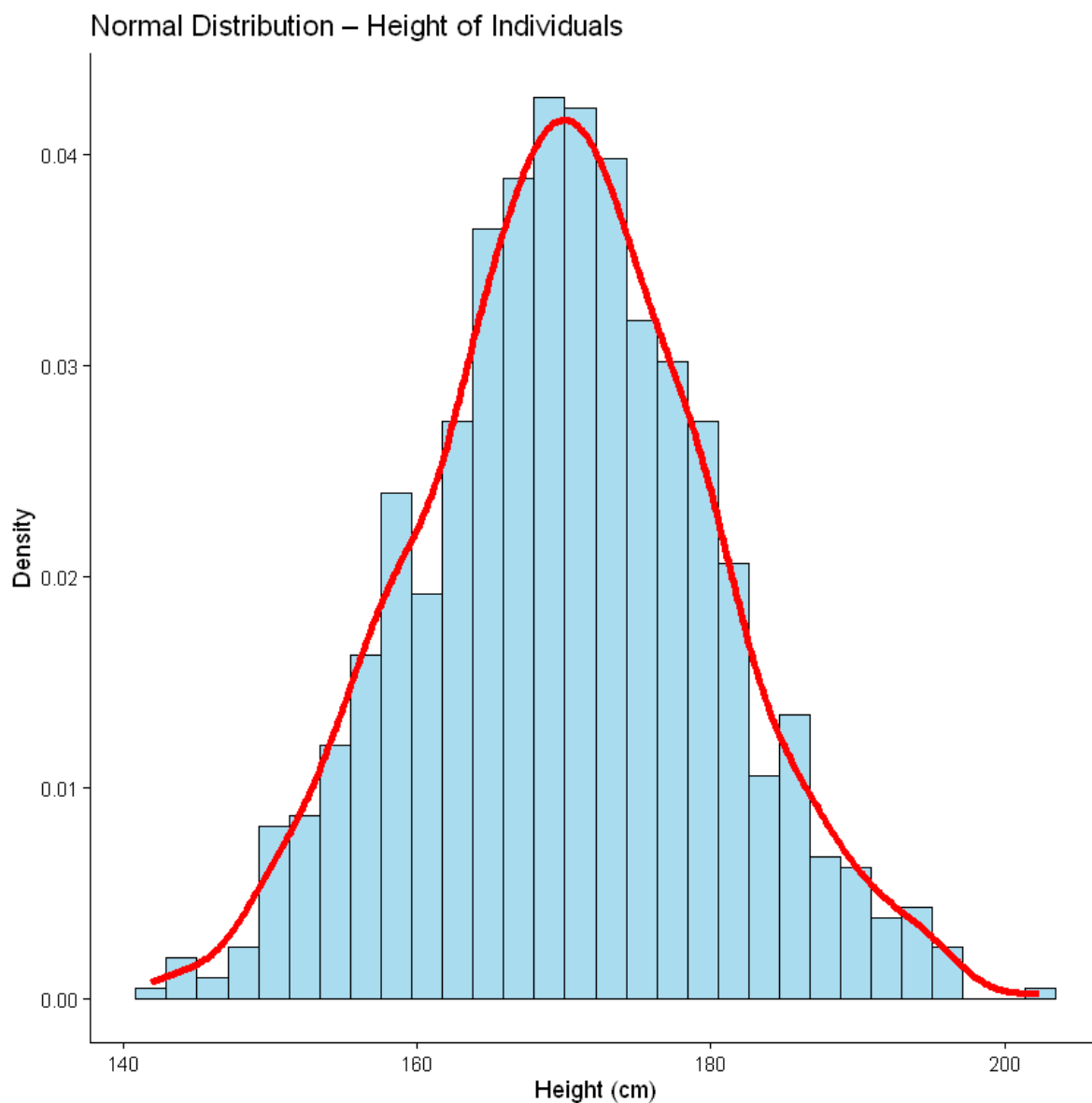


## Histogram and Density Curve (Normal Distribution)

LANGUAGE-R

```
Generate normal data
set.seed(123)
df_normal <- data.frame(
 values = rnorm(1000, mean = 170, sd = 10) # Height of individuals (cm)
)

ggplot(df_normal, aes(x = values)) +
 geom_histogram(aes(y = after_stat(density)),
 bins = 30,
 fill = "skyblue",
 color = "black",
 alpha = 0.7) +
 geom_density(color = "red", linewidth = 1.5) +
 labs(title = "Normal Distribution - Height of Individuals",
 x = "Height (cm)",
 y = "Density") +
 theme_classic()
```



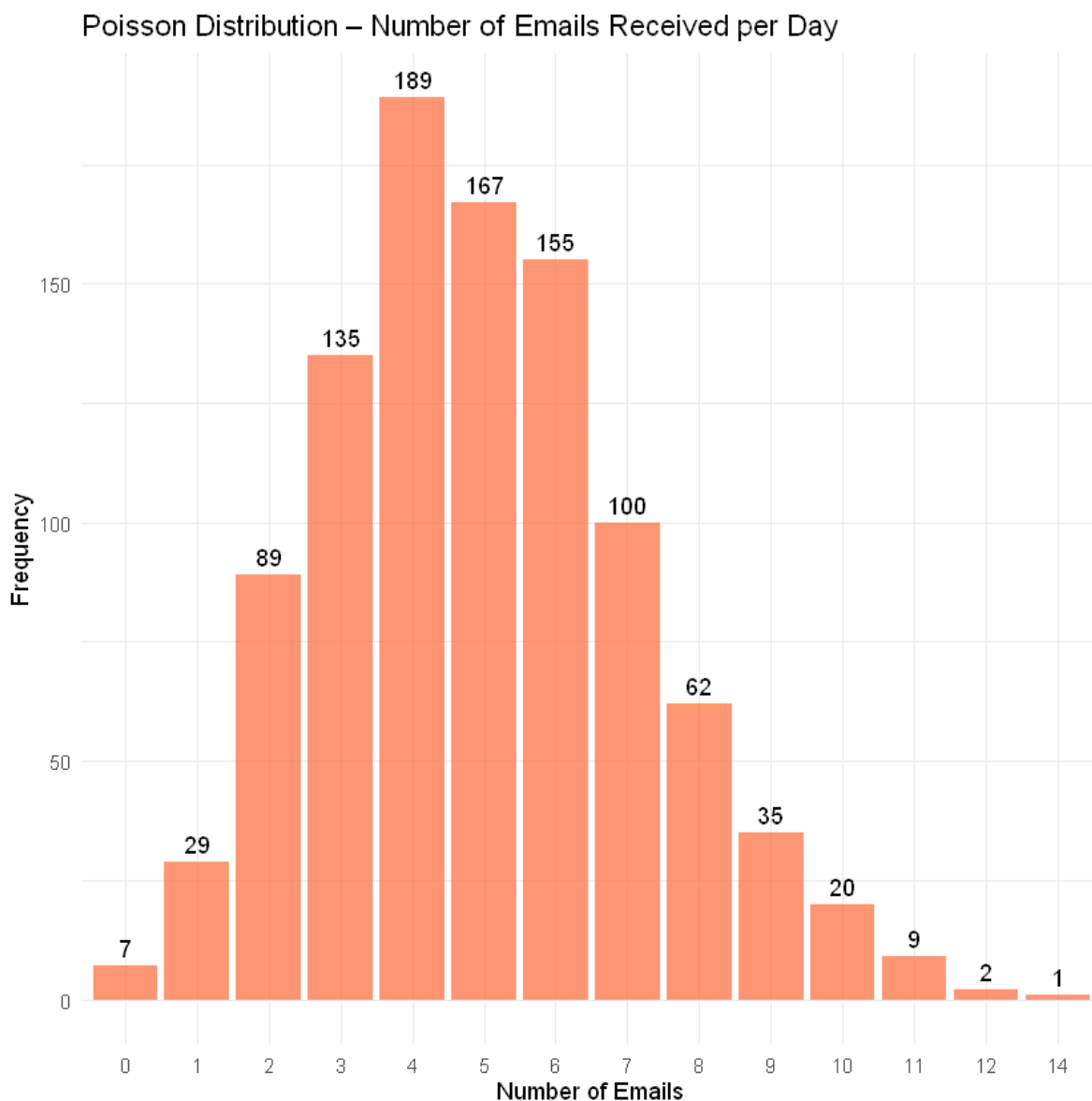
## Bar Chart (Poisson Distribution)

LANGUAGE-R

```
Poisson distribution (e.g., number of emails received per day)
set.seed(123)
df_poisson <- data.frame(
 emails = rpois(1000, lambda = 5)
)

Calculate frequencies
email_freq <- as.data.frame(table(df_poisson$emails))
names(email_freq) <- c("count", "frequency")

ggplot(email_freq, aes(x = count, y = frequency)) +
 geom_col(fill = "coral", alpha = 0.8) +
 geom_text(aes(label = frequency), vjust = -0.5) +
 labs(title = "Poisson Distribution - Number of Emails Received per Day",
 x = "Number of Emails",
 y = "Frequency") +
 theme_minimal()
```

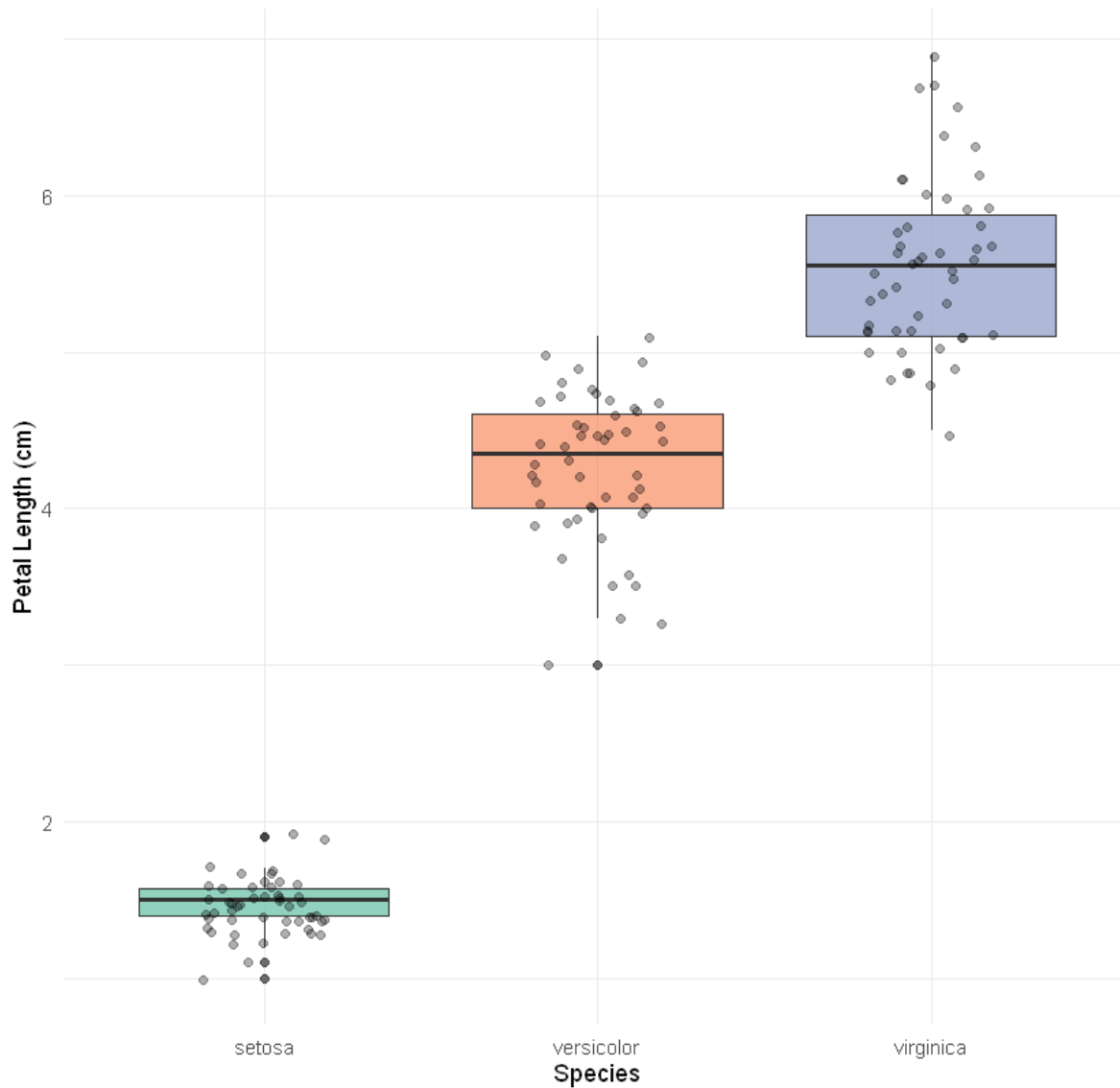


## Boxplot (Comparing Groups)

LANGUAGE-R

```
iris dataset
ggplot(iris, aes(x = Species, y = Petal.Length, fill = Species)) +
 geom_boxplot(alpha = 0.7) +
 geom_jitter(width = 0.2, alpha = 0.3) +
 labs(title = "Comparison of Petal Length Across Iris Species",
 x = "Species",
 y = "Petal Length (cm)") +
 scale_fill_brewer(palette = "Set2") +
 theme_minimal() +
 theme(legend.position = "none")
```

Comparison of Petal Length Across Iris Species



## Time Series Plot

LANGUAGE-R

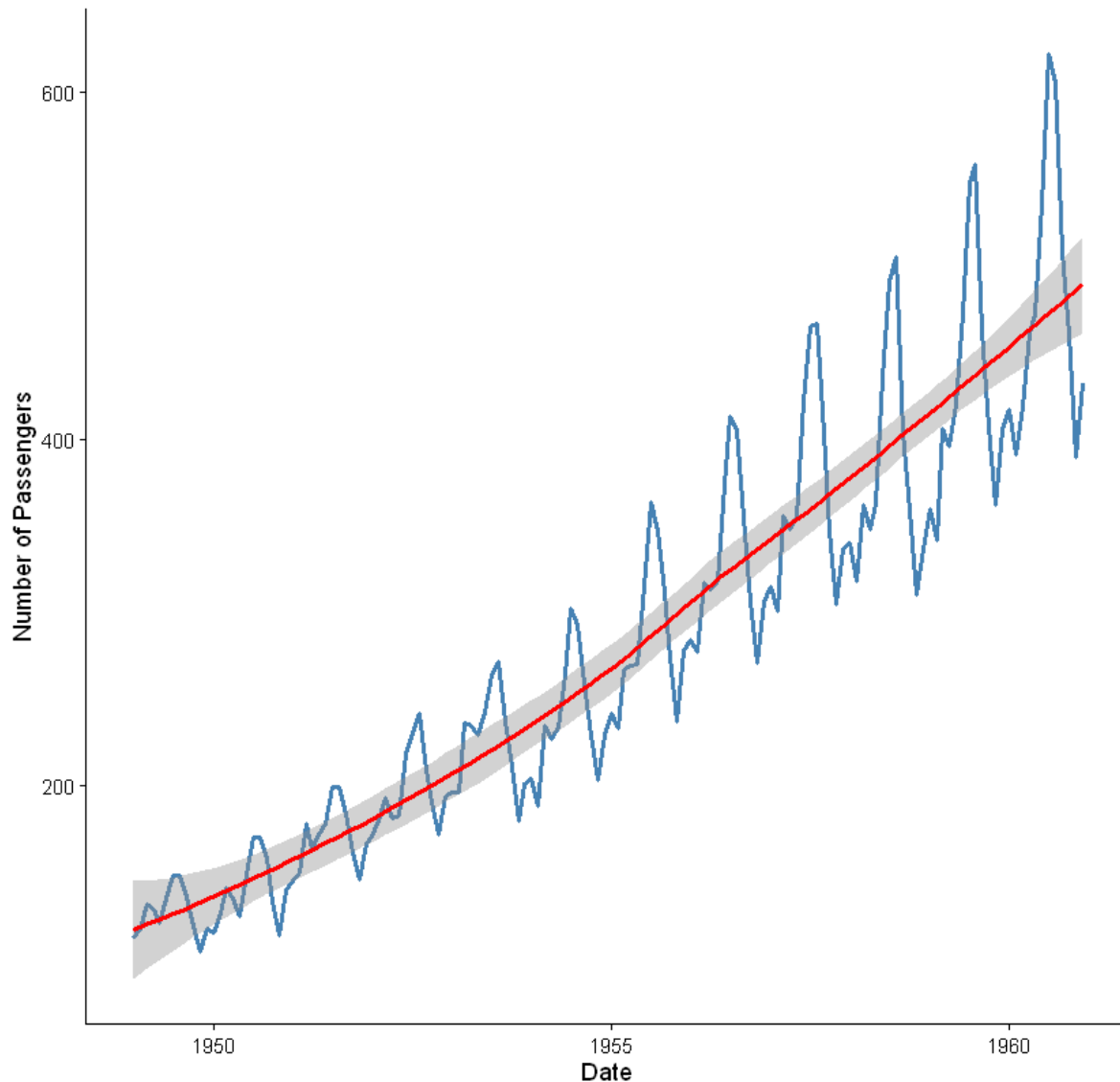
```
install.packages("zoo")
library(zoo) # for as.yearmon()

AirPassengers dataset
df_air <- data.frame(
 date = as.Date(as.yearmon(time(AirPassengers))),
 passengers = as.numeric(AirPassengers)
)

ggplot(df_air, aes(x = date, y = passengers)) +
 geom_line(color = "steelblue", linewidth = 1) +
 geom_smooth(method = "loess", color = "red", se = TRUE) +
 labs(title = "Airline Passengers Trend (1949-1960)",
 x = "Date",
 y = "Number of Passengers") +
 theme_classic()
```

geom\_smooth() using formula = 'y ~ x'

Airline Passengers Trend (1949-1960)



## More Examples

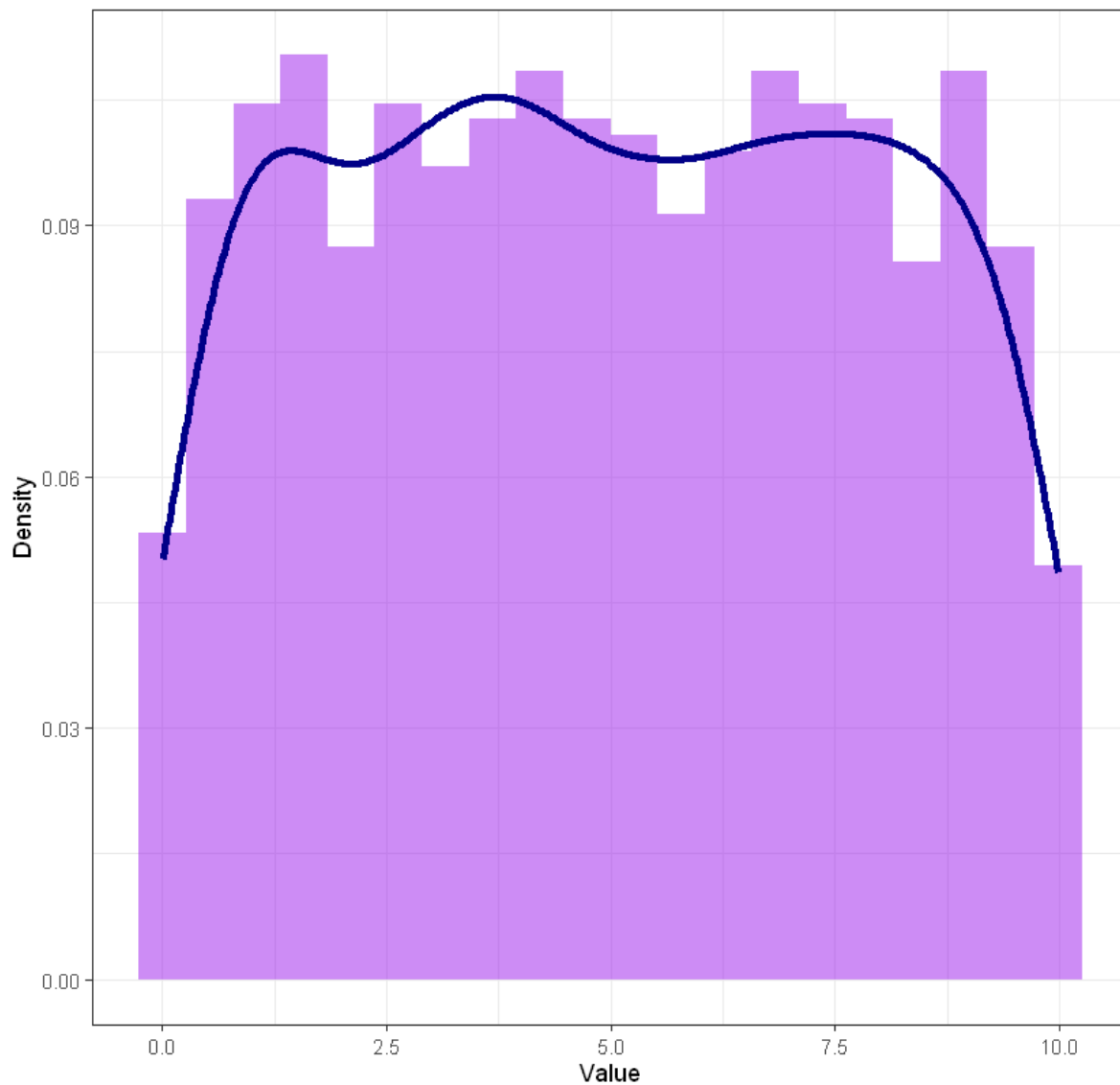
### Uniform Distribution

LANGUAGE-R

```
Uniform distribution (e.g., random number selection)
set.seed(123)
df_uniform <- data.frame(
 values = runif(1000, min = 0, max = 10)
)

ggplot(df_uniform, aes(x = values)) +
 geom_histogram(aes(y = after_stat(density)),
 bins = 20,
 fill = "purple",
 alpha = 0.5) +
 geom_density(color = "darkblue", linewidth = 1.5) +
 labs(title = "Uniform Distribution",
 x = "Value",
 y = "Density") +
 theme_bw()
```

Uniform Distribution



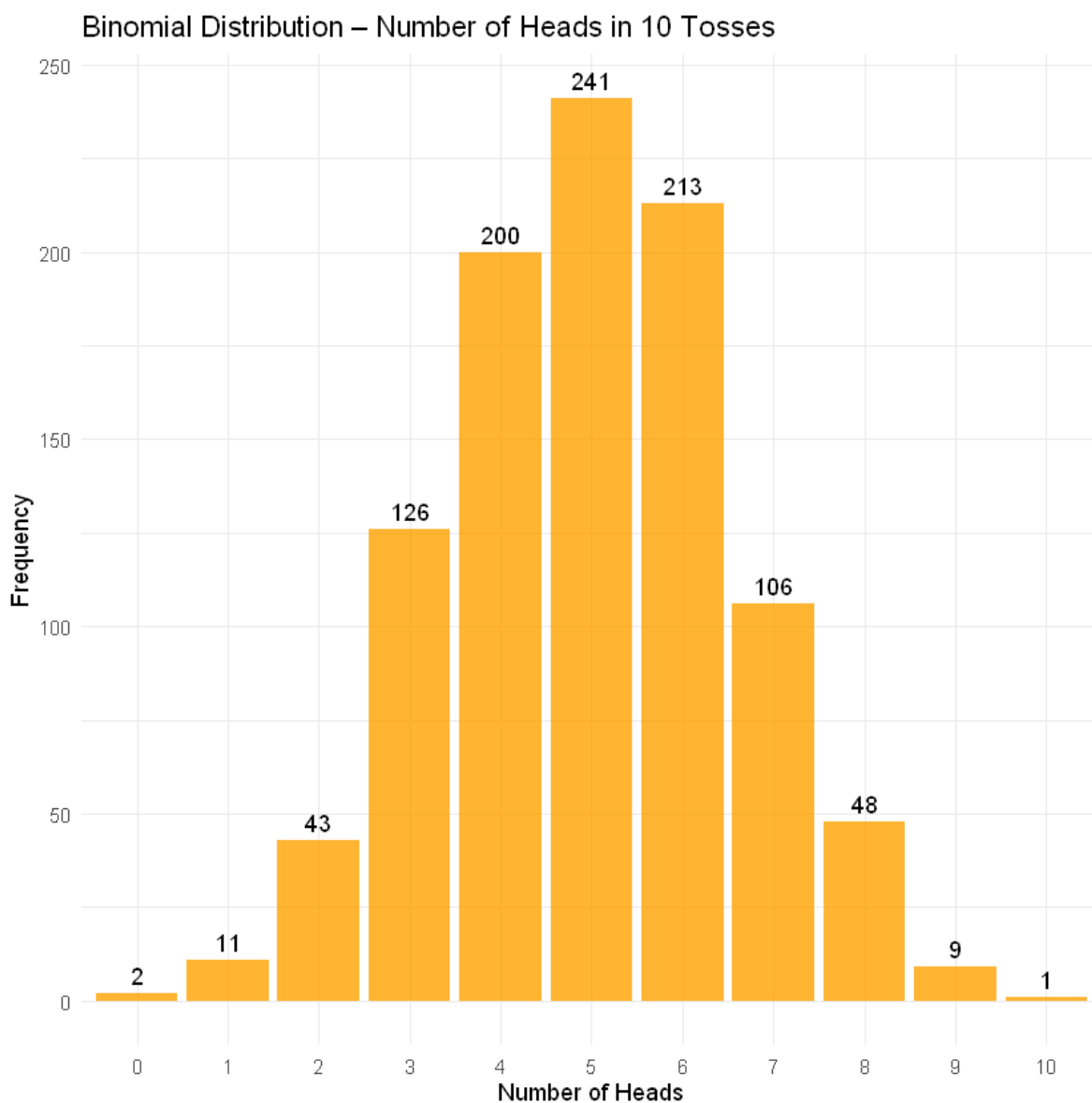
## Binomial Distribution

LANGUAGE-R

```
Binomial distribution (e.g., number of heads in 10 coin tosses)
set.seed(123)
df_binom <- data.frame(
 heads = rbinom(1000, size = 10, prob = 0.5)
)

binom_freq <- as.data.frame(table(df_binom$heads))
names(binom_freq) <- c("heads", "frequency")

ggplot(binom_freq, aes(x = heads, y = frequency)) +
 geom_col(fill = "orange", alpha = 0.8) +
 geom_text(aes(label = frequency), vjust = -0.5) +
 labs(title = "Binomial Distribution - Number of Heads in 10 Tosses",
 x = "Number of Heads",
 y = "Frequency") +
 theme_minimal()
```





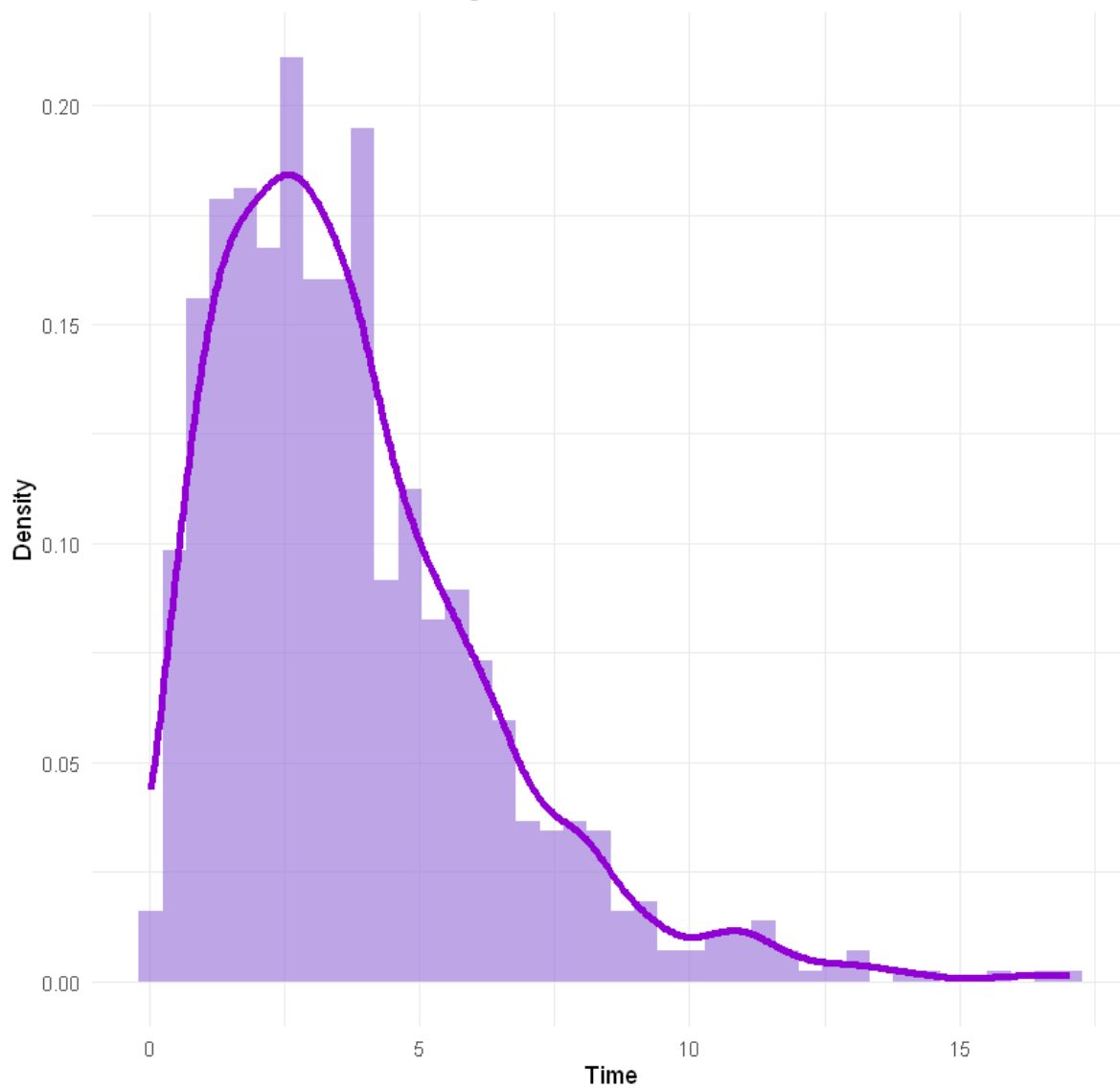
# Gamma Distribution

LANGUAGE-R

```
Gamma distribution (e.g., waiting time for consecutive events)
set.seed(123)
df_gamma <- data.frame(
 wait_time = rgamma(1000, shape = 2, rate = 0.5)
)

ggplot(df_gamma, aes(x = wait_time)) +
 geom_histogram(aes(y = after_stat(density)),
 bins = 40,
 fill = "mediumpurple",
 alpha = 0.6) +
 geom_density(color = "darkviolet", linewidth = 1.5) +
 labs(title = "Gamma Distribution - Waiting Time",
 x = "Time",
 y = "Density") +
 theme_minimal()
```

Gamma Distribution – Waiting Time



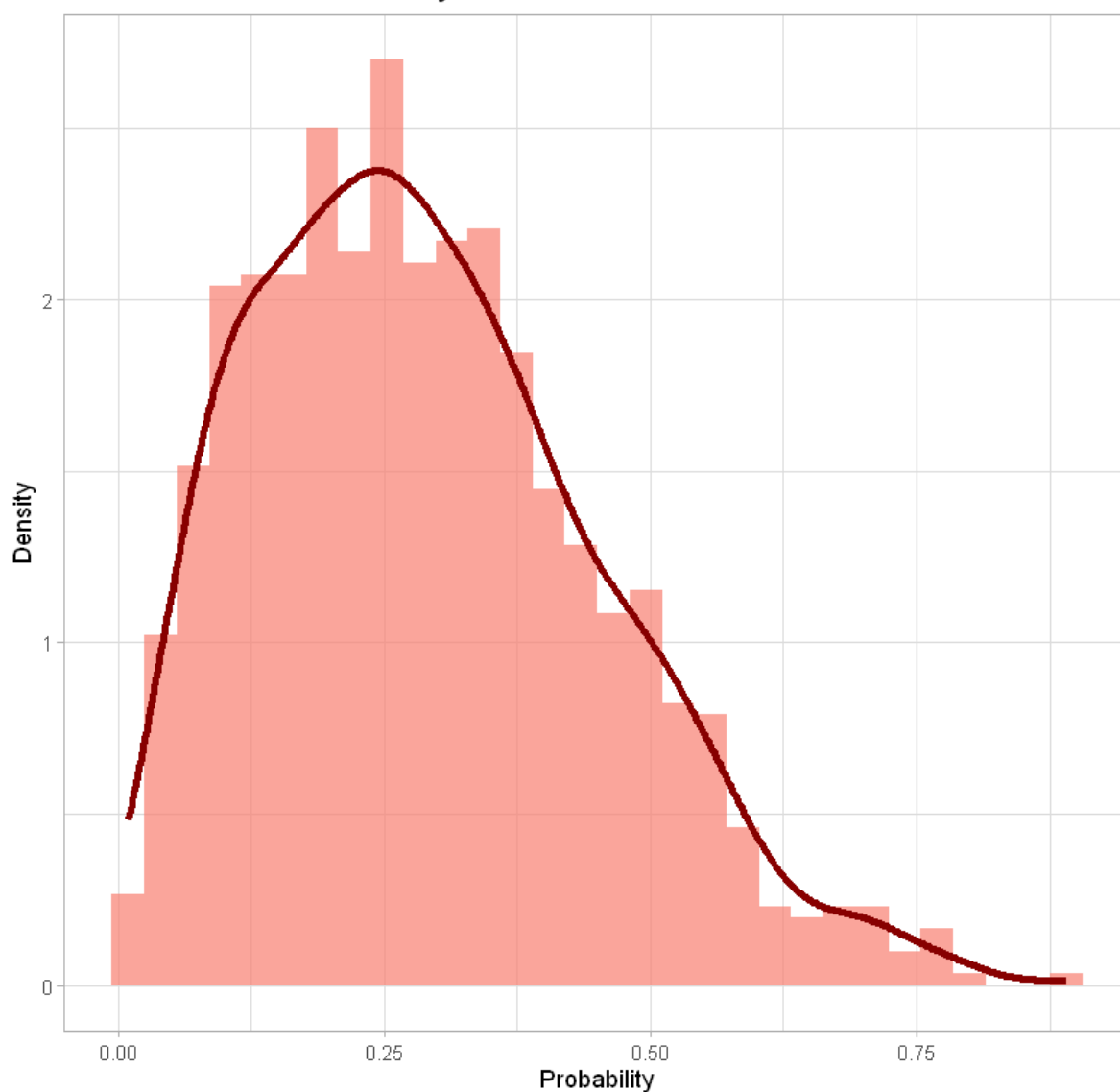
# Beta Distribution

LANGUAGE-R

```
Beta distribution (e.g., success rate, probabilities between 0 and 1)
set.seed(123)
df_beta <- data.frame(
 probability = rbeta(1000, shape1 = 2, shape2 = 5)
)

ggplot(df_beta, aes(x = probability)) +
 geom_histogram(aes(y = after_stat(density)),
 bins = 30,
 fill = "salmon",
 alpha = 0.7) +
 geom_density(color = "darkred", linewidth = 1.5) +
 labs(title = "Beta Distribution - Probability Distribution",
 x = "Probability",
 y = "Density") +
 theme_light()
```

Beta Distribution – Probability Distribution

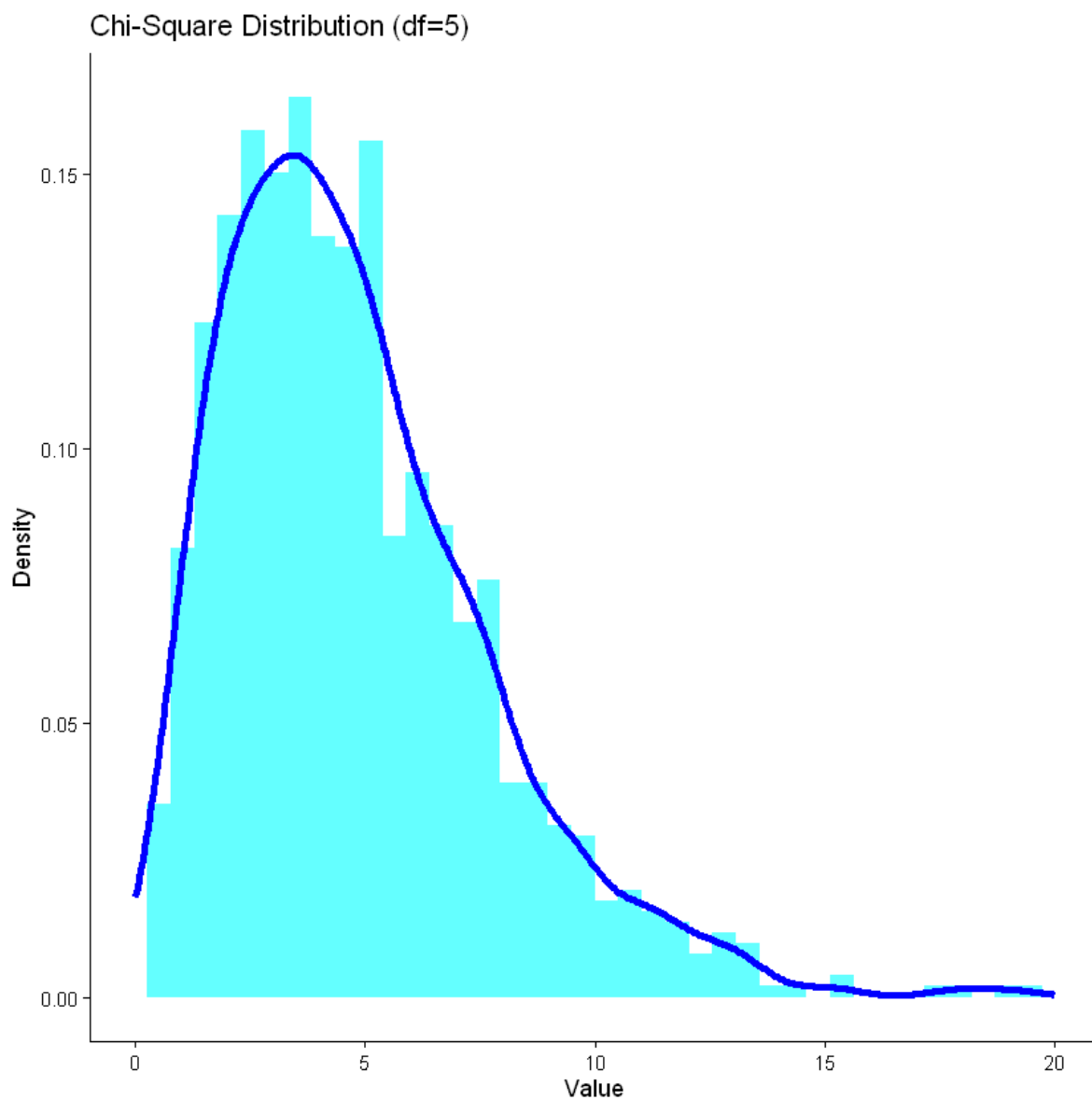


# Chi-Square Distribution

LANGUAGE-R

```
Chi-Square distribution (used in statistical tests)
set.seed(123)
df_chisq <- data.frame(
 values = rchisq(1000, df = 5)
)

ggplot(df_chisq, aes(x = values)) +
 geom_histogram(aes(y = after_stat(density)),
 bins = 40,
 fill = "cyan",
 alpha = 0.6) +
 geom_density(color = "blue", linewidth = 1.5) +
 xlim(0, 20) +
 labs(title = "Chi-Square Distribution (df=5)",
 x = "Value",
 y = "Density") +
 theme_classic()
```



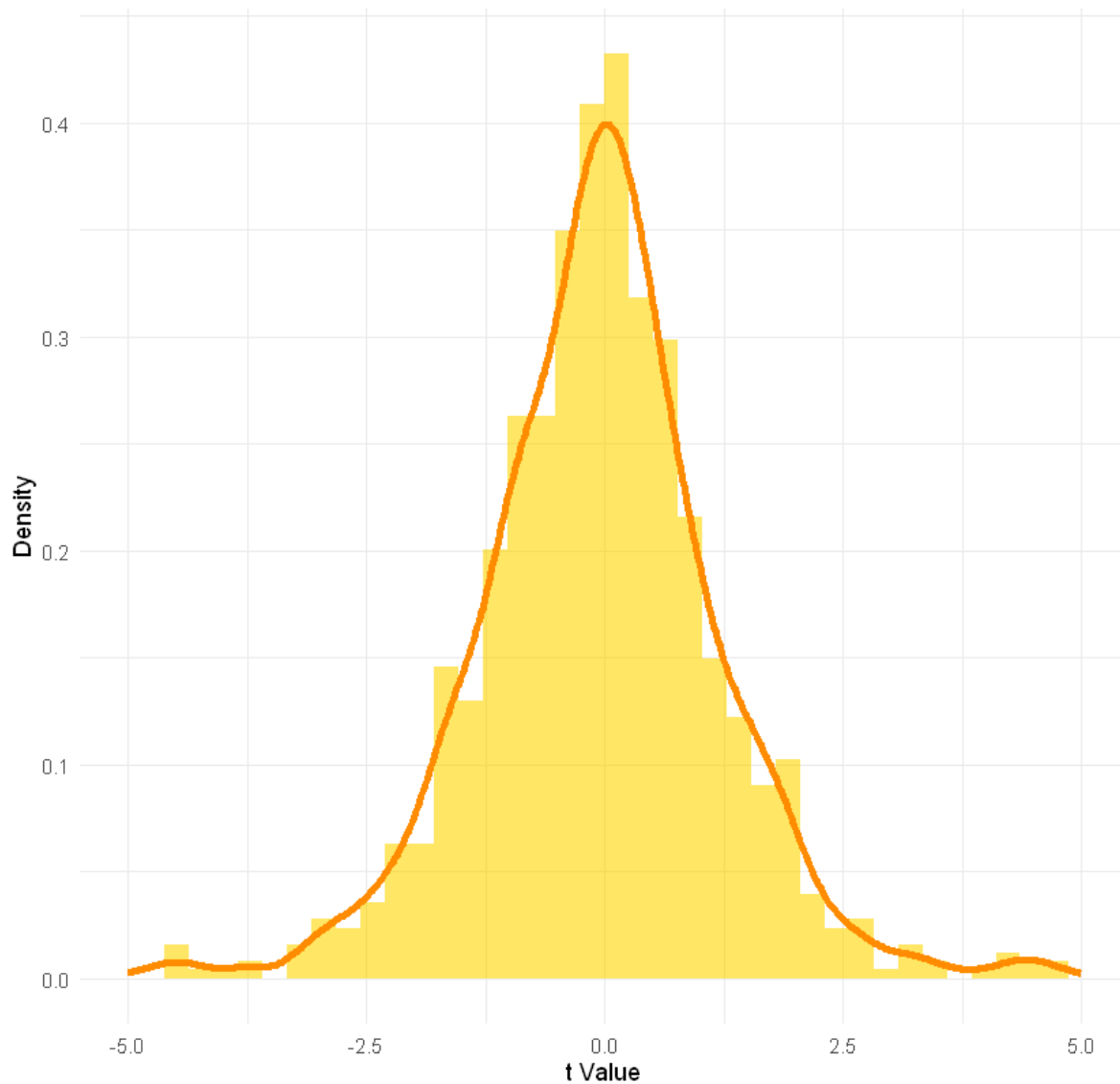
## Student's t Distribution

LANGUAGE-R

```
Student's t distribution (used in t-tests)
set.seed(123)
df_t <- data.frame(
 t_values = rt(1000, df = 5)
)

ggplot(df_t, aes(x = t_values)) +
 geom_histogram(aes(y = after_stat(density)),
 bins = 40,
 fill = "gold",
 alpha = 0.6) +
 geom_density(color = "darkorange", linewidth = 1.5) +
 xlim(-5, 5) +
 labs(title = "Student's t Distribution (df=5)",
 x = "t Value",
 y = "Density") +
 theme_minimal()
```

Student's t Distribution (df=5)

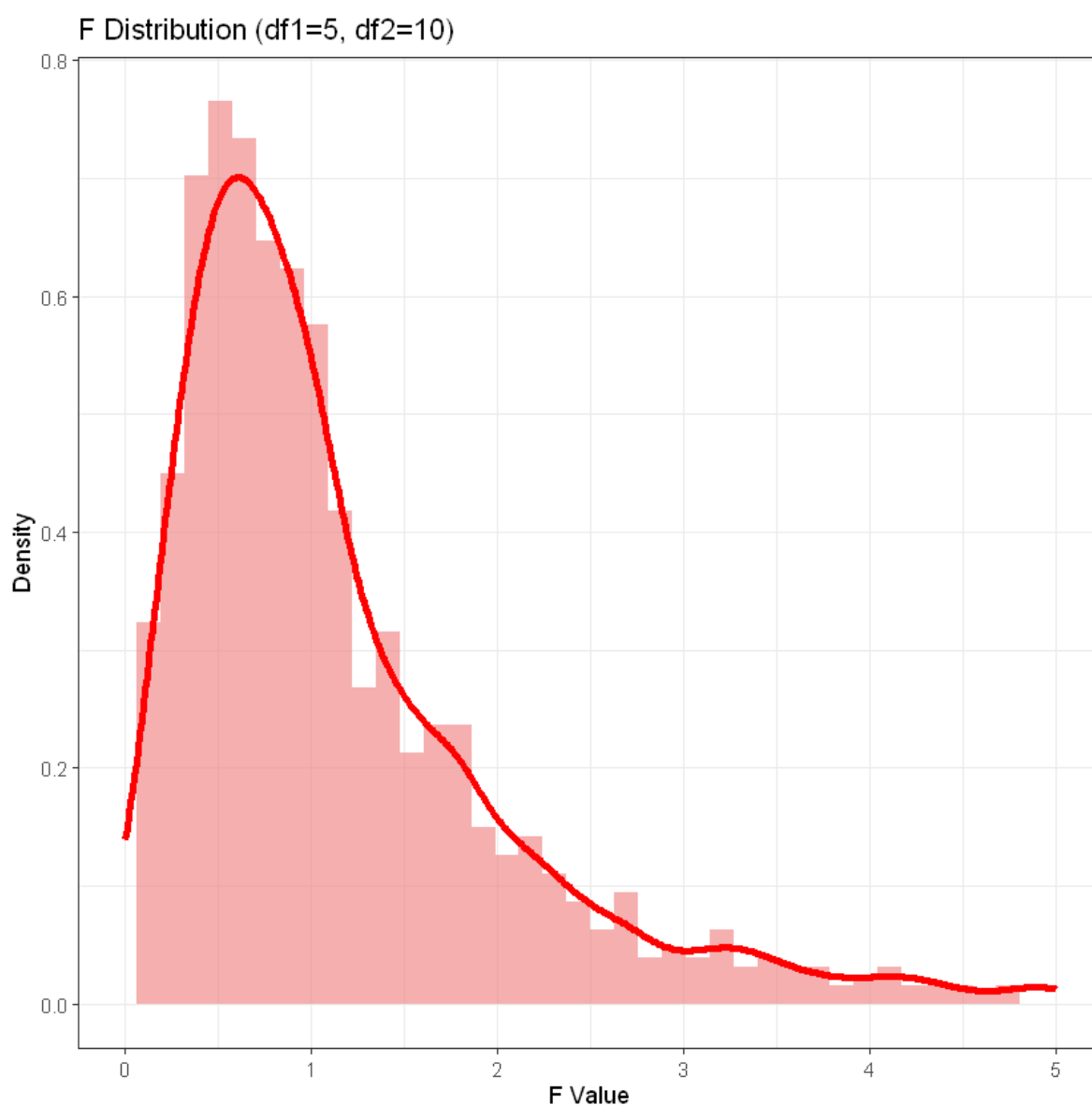


## F Distribution

LANGUAGE-R

```
F distribution (used in ANOVA)
set.seed(123)
df_f <- data.frame(
 f_values = rf(1000, df1 = 5, df2 = 10)
)

ggplot(df_f, aes(x = f_values)) +
 geom_histogram(aes(y = after_stat(density)),
 bins = 40,
 fill = "lightcoral",
 alpha = 0.6) +
 geom_density(color = "red", linewidth = 1.5) +
 xlim(0, 5) +
 labs(title = "F Distribution (df1=5, df2=10)",
 x = "F Value",
 y = "Density") +
 theme_bw()
```



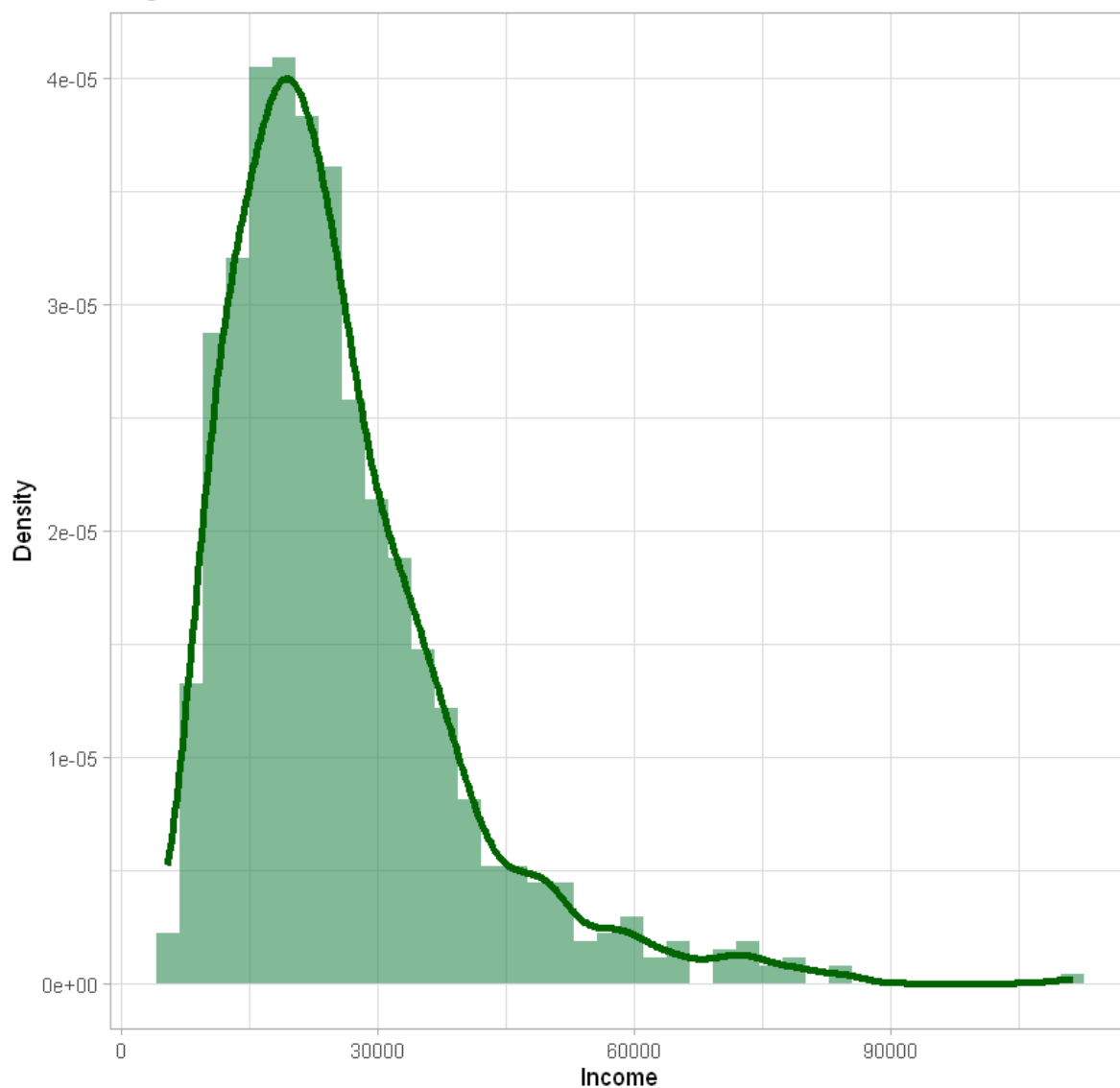
# Log-Normal Distribution

LANGUAGE-R

```
Log-normal distribution (e.g., income, stock prices)
set.seed(123)
df_lognorm <- data.frame(
 income = rlnorm(1000, meanlog = 10, sdlog = 0.5)
)

ggplot(df_lognorm, aes(x = income)) +
 geom_histogram(aes(y = after_stat(density)),
 bins = 40,
 fill = "seagreen",
 alpha = 0.6) +
 geom_density(color = "darkgreen", linewidth = 1.5) +
 labs(title = "Log-Normal Distribution - Income",
 x = "Income",
 y = "Density") +
 theme_light()
```

Log-Normal Distribution – Income



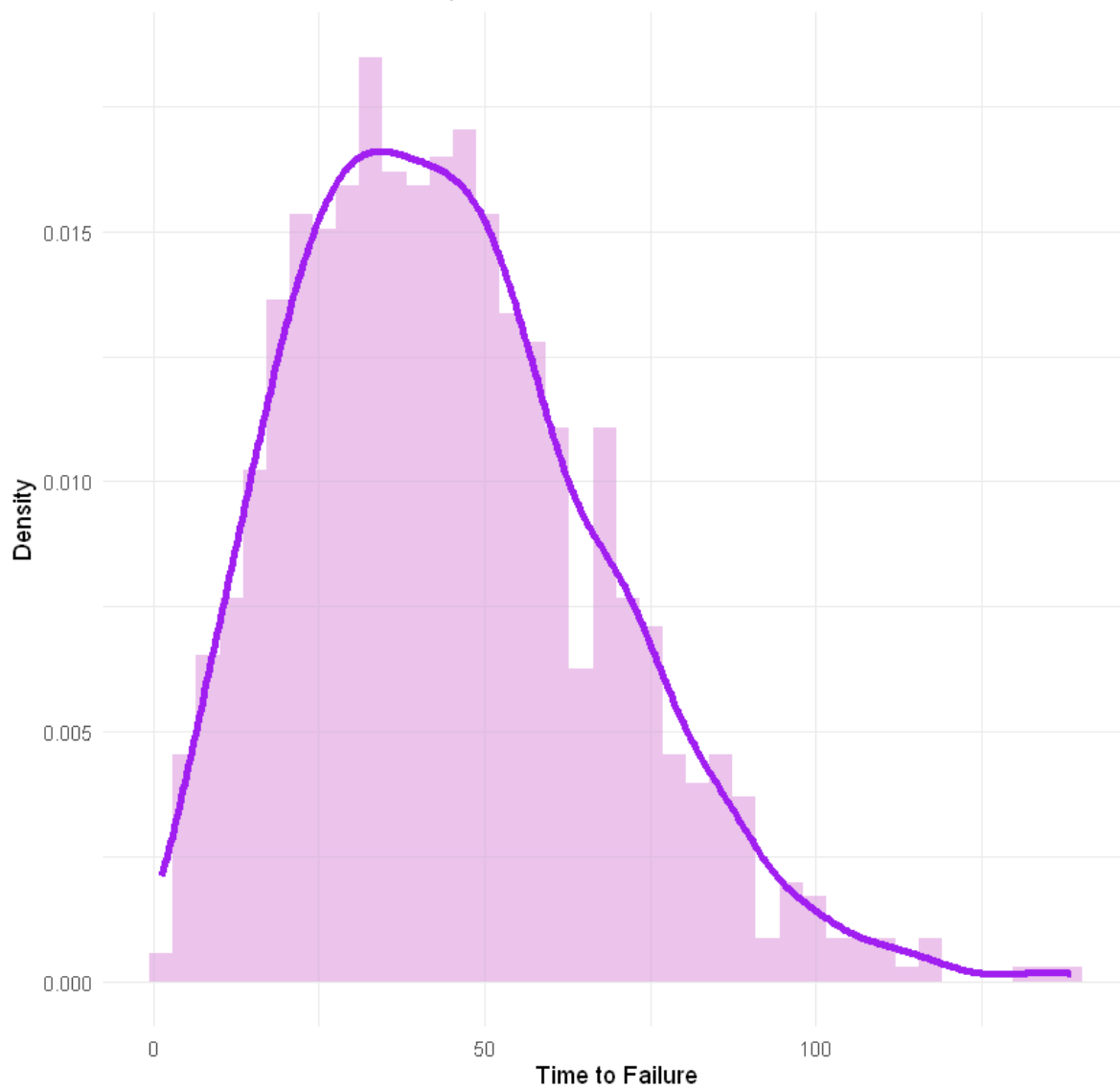
# Weibull Distribution

LANGUAGE-R

```
Weibull distribution (e.g., component lifetimes, time to failure)
set.seed(123)
df_weibull <- data.frame(
 lifetime = rweibull(1000, shape = 2, scale = 50)
)

ggplot(df_weibull, aes(x = lifetime)) +
 geom_histogram(aes(y = after_stat(density)),
 bins = 40,
 fill = "plum",
 alpha = 0.6) +
 geom_density(color = "purple", linewidth = 1.5) +
 labs(title = "Weibull Distribution - Component Lifetimes",
 x = "Time to Failure",
 y = "Density") +
 theme_minimal()
```

Weibull Distribution – Component Lifetimes



## Geometric Distribution

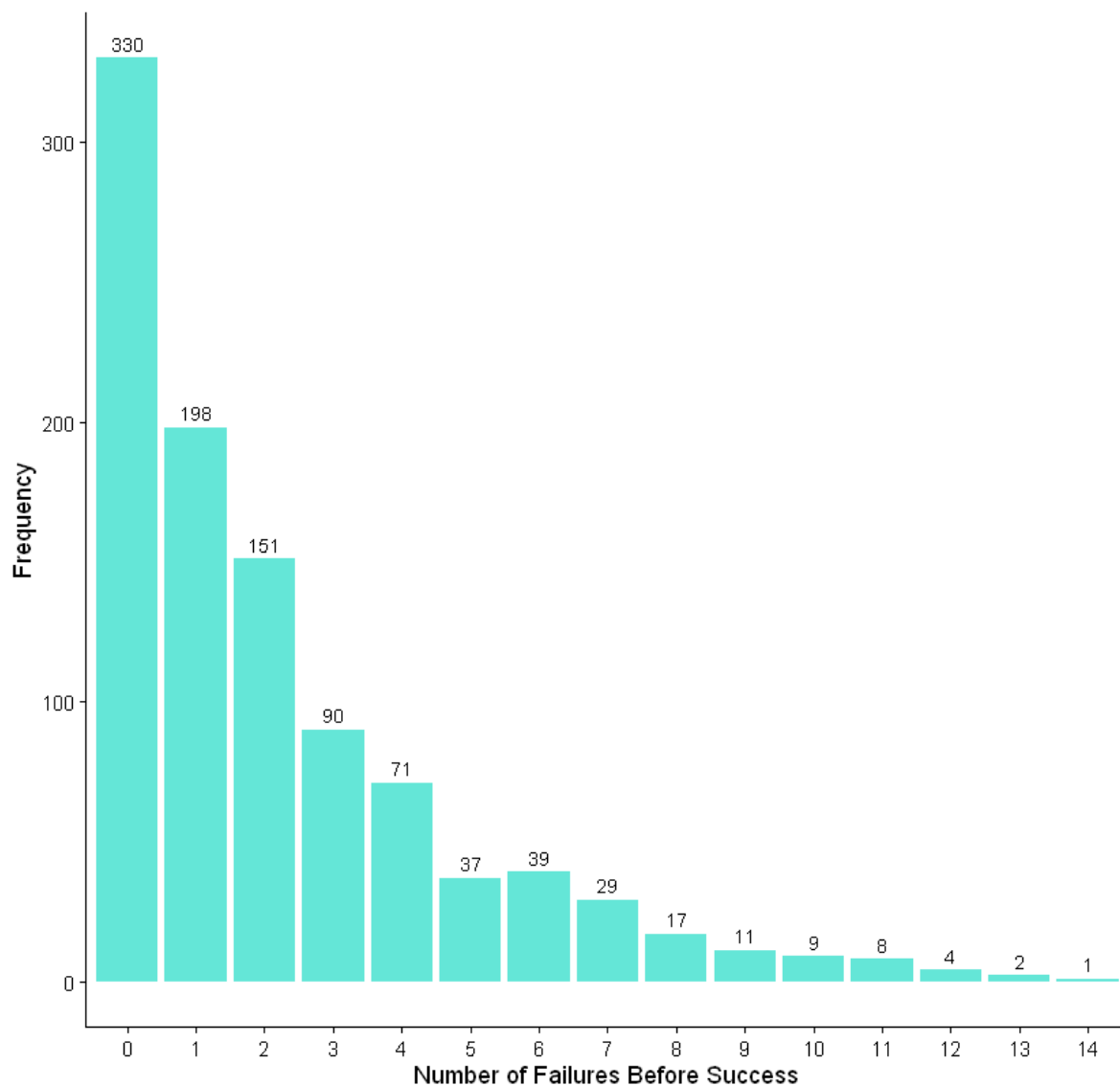
LANGUAGE-R

```
Geometric distribution (number of trials until first success)
set.seed(123)
df_geom <- data.frame(
 trials = rgeom(1000, prob = 0.3)
)

geom_freq <- as.data.frame(table(df_geom$trials))
names(geom_freq) <- c("trials", "frequency")

ggplot(geom_freq[1:15,], aes(x = trials, y = frequency)) +
 geom_col(fill = "turquoise", alpha = 0.8) +
 geom_text(aes(label = frequency), vjust = -0.5, size = 3) +
 labs(title = "Geometric Distribution - Number of Trials Until First Success",
 x = "Number of Failures Before Success",
 y = "Frequency") +
 theme_classic()
```

Geometric Distribution – Number of Trials Until First Success





# Negative Binomial Distribution

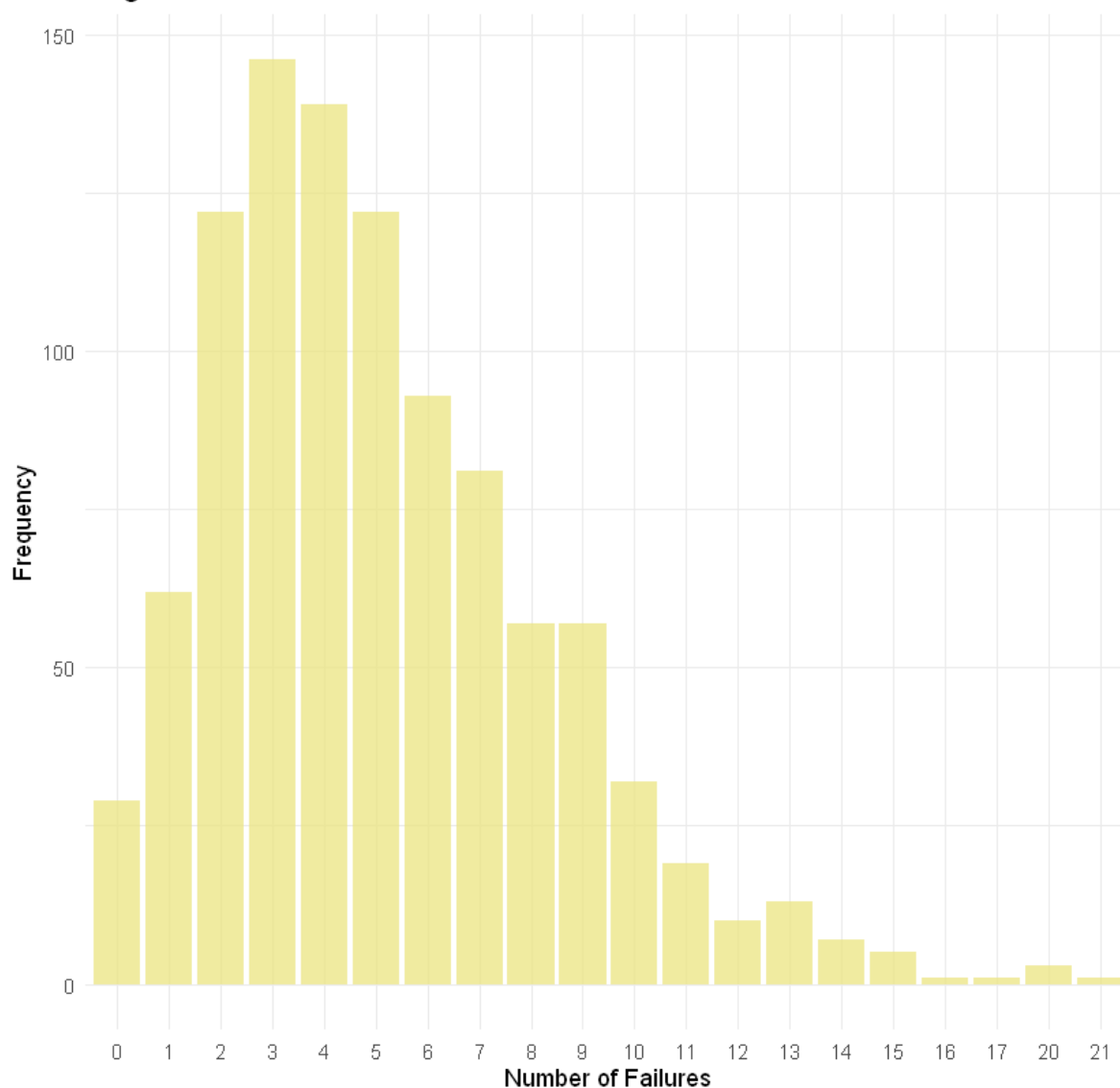
LANGUAGE-R

```
Negative Binomial distribution (number of failures before r successes)
set.seed(123)
df_nbinom <- data.frame(
 failures = rnbinom(1000, size = 5, prob = 0.5)
)

nbinom_freq <- as.data.frame(table(df_nbinom$failures))
names(nbinom_freq) <- c("failures", "frequency")

ggplot(nbinom_freq[1:20,], aes(x = failures, y = frequency)) +
 geom_col(fill = "khaki", alpha = 0.8) +
 labs(title = "Negative Binomial Distribution",
 x = "Number of Failures",
 y = "Frequency") +
 theme_minimal()
```

Negative Binomial Distribution



# Cauchy Distribution

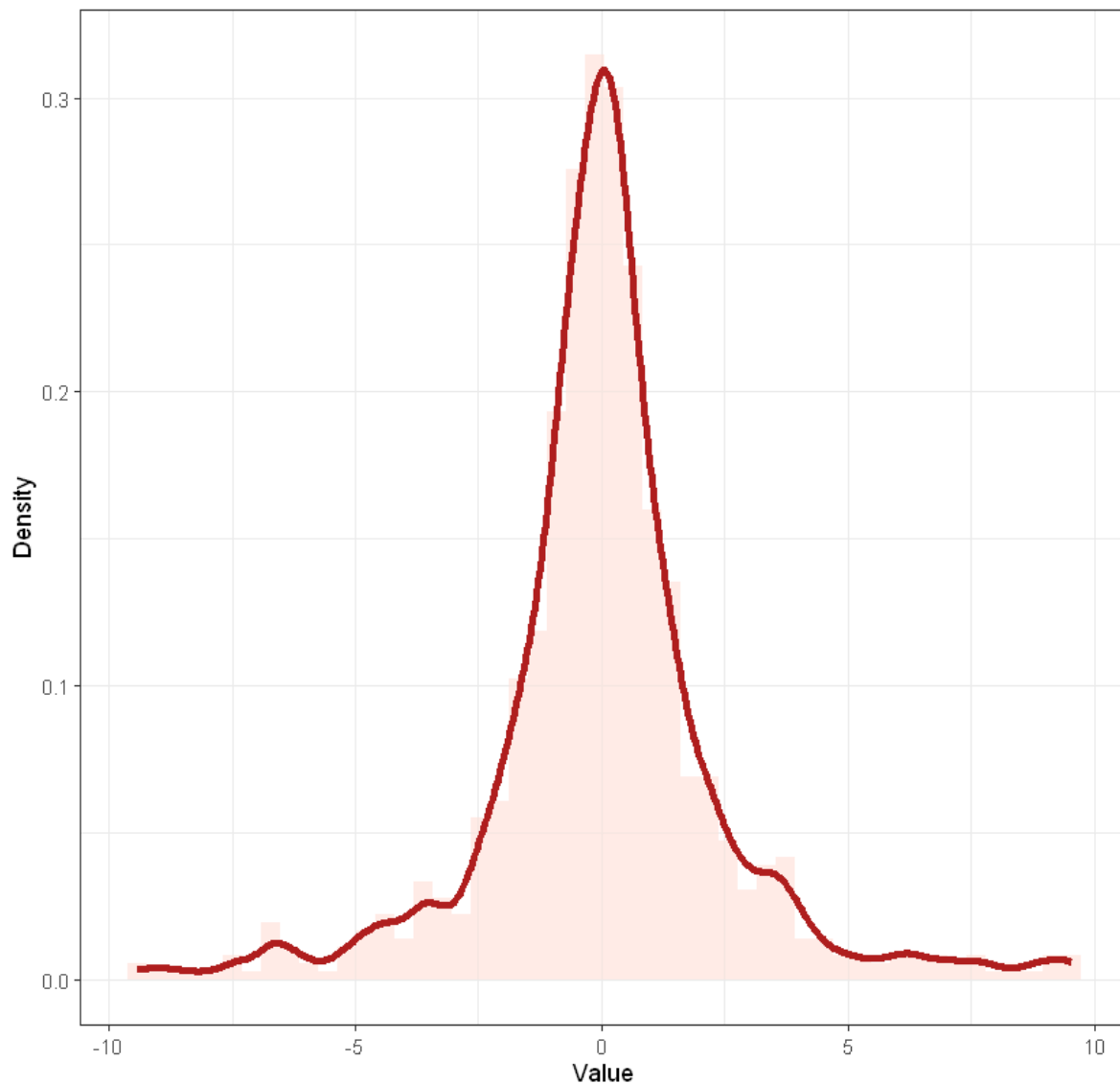
LANGUAGE-R

```
Cauchy distribution (heavy-tailed)
set.seed(123)
df_cauchy <- data.frame(
 values = rcauchy(1000, location = 0, scale = 1)
)

Filter extreme values and ensure it remains a data.frame
df_cauchy_filtered <- df_cauchy[df_cauchy$values > -10 & df_cauchy$values < 10, , drop = FALSE]

ggplot(df_cauchy_filtered, aes(x = values)) +
 geom_histogram(aes(y = after_stat(density)),
 bins = 50,
 fill = "mistyrose",
 alpha = 0.7) +
 geom_density(color = "firebrick", linewidth = 1.5) +
 labs(title = "Cauchy Distribution - Heavy-Tailed Distribution",
 x = "Value",
 y = "Density") +
 theme_bw()
```

Cauchy Distribution – Heavy-Tailed Distribution



# Comparing Different Distributions in a Single Plot

## Comparison of Continuous Distributions

LANGUAGE-R

```
install.packages("tidyr")
install.packages("dplyr")

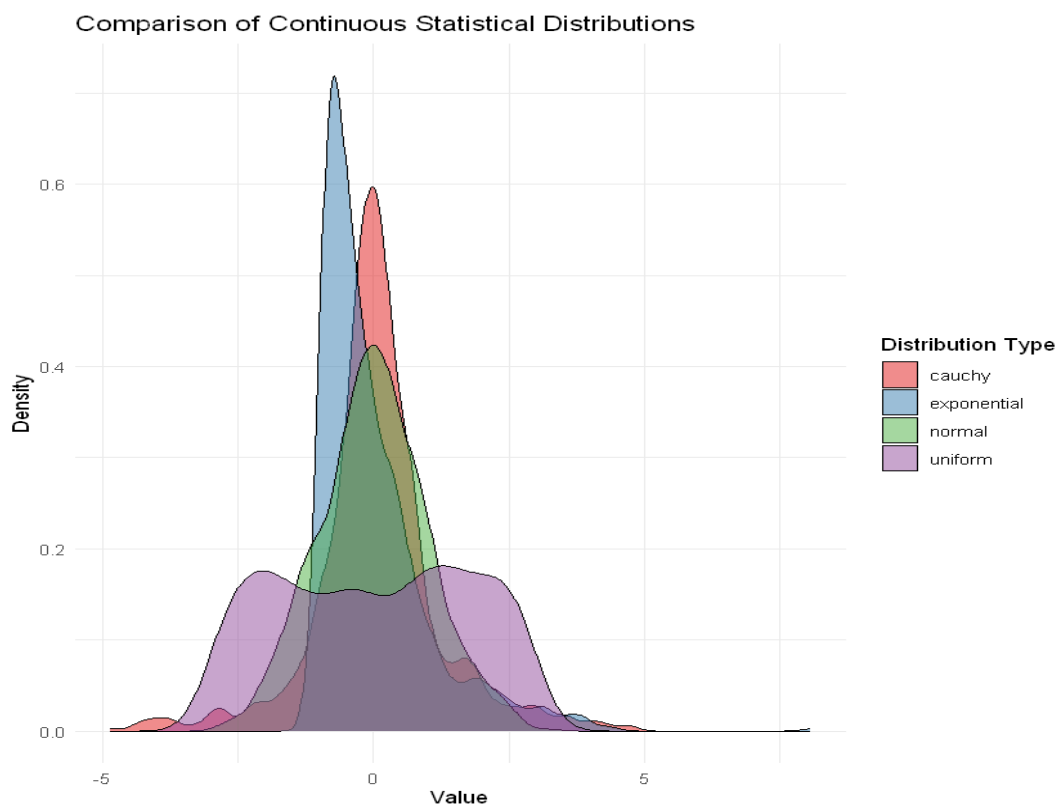
library(tidyr)
library(dplyr)
library(ggplot2)

set.seed(123)
df_compare <- data.frame(
 normal = rnorm(1000, 0, 1),
 uniform = runif(1000, -3, 3),
 exponential = rexp(1000, 1) - 1,
 cauchy = rcauchy(1000, 0, 0.5)
)

Filter extreme values for better visualization
df_compare_filtered <- df_compare %>%
 filter(cauchy > -5 & cauchy < 5)

df_long <- df_compare_filtered %>%
 pivot_longer(cols = everything(),
 names_to = "distribution",
 values_to = "value")

ggplot(df_long, aes(x = value, fill = distribution)) +
 geom_density(alpha = 0.5) +
 labs(title = "Comparison of Continuous Statistical Distributions",
 x = "Value",
 y = "Density",
 fill = "Distribution Type") +
 theme_minimal() +
 scale_fill_brewer(palette = "Set1")
```



# Comparison of Different Normal Distributions

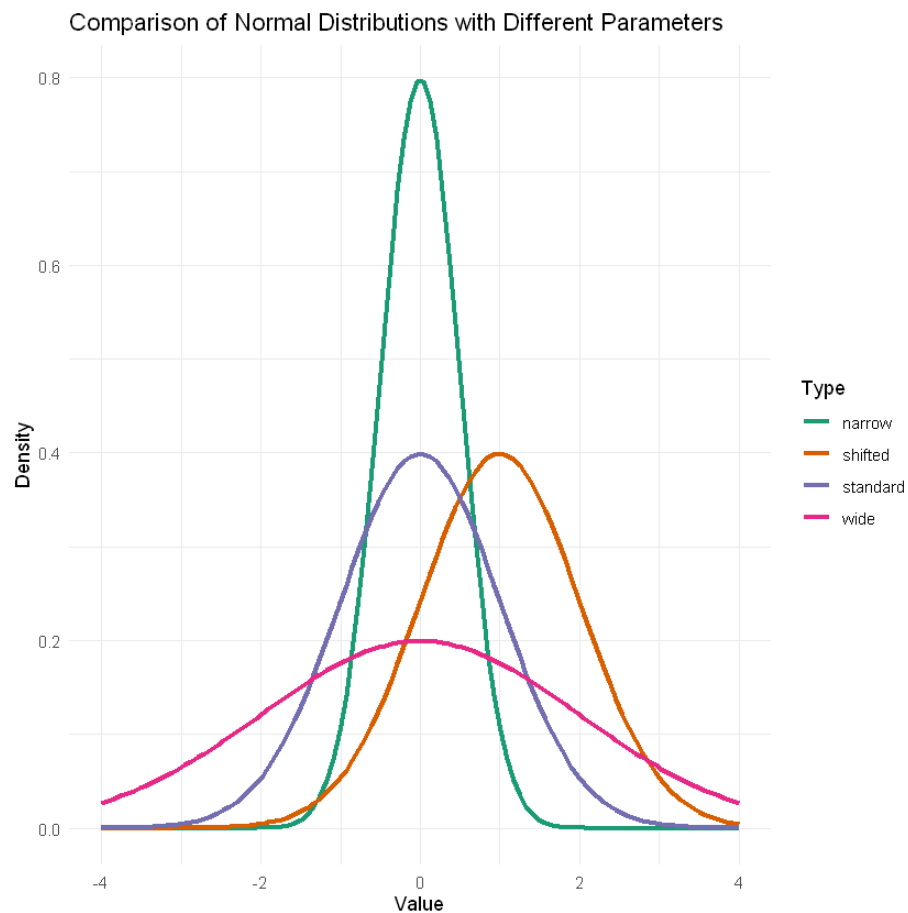
LANGUAGE-R

```
set.seed(123)
df_normal_compare <- data.frame(
 x = seq(-4, 4, length.out = 100)
)

df_normal_compare <- df_normal_compare %>%
 mutate(
 standard = dnorm(x, 0, 1),
 wide = dnorm(x, 0, 2),
 narrow = dnorm(x, 0, 0.5),
 shifted = dnorm(x, 1, 1)
)

df_norm_long <- df_normal_compare %>%
 pivot_longer(cols = -x, names_to = "type", values_to = "density")

ggplot(df_norm_long, aes(x = x, y = density, color = type)) +
 geom_line(linewidth = 1.2) +
 labs(title = "Comparison of Normal Distributions with Different Parameters",
 x = "Value",
 y = "Density",
 color = "Type") +
 theme_minimal() +
 scale_color_brewer(palette = "Dark2")
```



# Summary of Statistical Distributions

Distribution	Type	Applications
Normal	Continuous	Height, weight, errors
Uniform	Continuous	Random selection
Exponential	Continuous	Time between events
Poisson	Discrete	Number of events in a time period
Binomial	Discrete	Number of successes
Gamma	Continuous	Waiting time for multiple events
Beta	Continuous	Probabilities, rates
Chi-Square	Continuous	Independence tests
t-Student	Continuous	Mean testing
F	Continuous	ANOVA
Log-Normal	Continuous	Income, stock prices
Weibull	Continuous	Component lifetimes
Geometric	Discrete	Trials until first success
Negative Binomial	Discrete	Failures until r successes
Cauchy	Continuous	Physics, finance

## Key Points

### Base R vs. ggplot2

- **Base R:** Fast and simple for basic plots
- **ggplot2:** Flexible, professional, and suitable for complex visualizations

### ggplot2 Themes

```
Different themes
theme_minimal() # Simple and clean
theme_classic() # Classic, no grid
theme_bw() # Black and white
theme_dark() # Dark
theme_light() # Light
```

LANGUAGE-R

### Saving Plots

```
Save the last ggplot2 plot
ggsave("my_plot.png", width = 10, height = 6, dpi = 300)

Save a Base R plot
png("my_base_plot.png", width = 800, height = 600)
plot(mtcars$wt, mtcars$mpg)
dev.off()
```

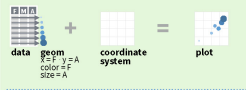
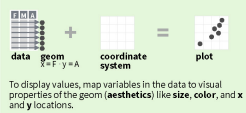
LANGUAGE-R

# Data visualization with ggplot2 : : CHEATSHEET



## Basics

ggplot2 is based on the **grammar of graphics**, the idea that you can build every graph from the same components: a **data set**, a **coordinate system**, and **geoms**—visual marks that represent data points.



Complete the template below to build a graph.

```
ggplot(data = <DATA>) +
 <GEOM FUNCTION>(mapping = aes(<MAPPINGS>))
 stat(<STAT>, position = <POSITION>)
 <COORDINATE FUNCTION>
 <FACET FUNCTION>
 <SCALE FUNCTION>
 <THEME FUNCTION>
```

Required  
Not required, sensible defaults supplied

ggplot(data = mpg, aes(x = city, y = hwy)) Begins a plot that you finish by adding layers to. Add one geom function per layer.

last\_plot() Returns the last plot.

ggsave("plot.png", width = 5, height = 5) Saves last plot as 5 x 5" file named "plot.png" in working directory. Matches file type to file extension.

## Aes

**Common aesthetic values.**  
color and fill - string ("red", "#RRGGBB")  
linetype - integer or string (0 = "blank", 1 = "solid", 2 = "dashed", 3 = "dotted", 4 = "dottedash", 5 = "longdash", 6 = "twodash")  
size - integer (in mm for size of points and text)  
linewidth - integer (in mm for width of lines)  
shape - integer/shape name or a single character ("a")



## Geoms

Use a geom function to represent data points, use the geom's aesthetic properties to represent variables. Each function returns a layer.

### GRAPHICAL PRIMITIVES

a <- geom\_blank() and a <- expand\_limits() Ensure limits include values across all plots.  
b <- geom\_curve(aes(yend = lat + 1, xend = long + 1), curvature = 1) - x, yend, y, yend, alpha, angle, color, curvature, linetype, size  
a <- geom\_path(linetype = "butt", linjoin = "round", linemite = 1) x, y, alpha, color, group, linetype, size  
a <- geom\_polygon(aes(alpha = 50)) - x, y, alpha, color, fill, group, subgroup, linetype, size  
b <- geom\_rect(aes(xmin = long, ymin = lat, xmax = long + 1, ymax = lat + 1)) - xmax, xmin, ymax, ymin, alpha, color, fill, linetype, size  
a <- geom\_ribbon(aes(ymin = unemployment - 900, ymax = unemployment + 900)) - x, ymax, ymin, alpha, color, fill, group, linetype, size

### LINE SEGMENTS

common aesthetics: x, y, alpha, color, linetype, size  
b <- geom\_abline(aes(intercept = 0, slope = 1))  
b <- geom\_vline(aes(intercept = lat))  
b <- geom\_segment(aes(yend = lat + 1, xend = long + 1))  
b <- geom\_spoke(aes(angle = 1:155, radius = 1))

### ONE VARIABLE continuous

c <- ggplot(mpg, aes(hwy)); c2 <- ggplot(mpg)  
c <- geom\_area(stat = "bin") x, y, alpha, color, fill, linetype, size  
c <- geom\_density(kernel = "gaussian") x, y, alpha, color, fill, group, linetype, size, weight  
c <- geom\_dotplot() x, y, alpha, color, fill  
c <- geom\_freqpoly() x, y, alpha, color, group, linetype, size  
c <- geom\_histogram(binwidth = 5) x, y, alpha, color, fill, linetype, size, weight  
c2 <- geom\_qq(aes(sample = hwy)) x, y, alpha, color, fill, linetype, size, weight

### discrete

d <- ggplot(mpg, aes(f))  
d <- geom\_bar() x, alpha, color, fill, linetype, size, weight

### TWO VARIABLES

both continuous  
e <- ggplot(mpg, aes(cty, hwy))  
e <- geom\_label(aes(label = cty)) - x, y, label, alpha, angle, color, family, fontface, hjust, linetype, size, vjust  
e <- geom\_point() x, y, alpha, color, fill, shape, size, stroke  
e <- geom\_quantile() x, y, alpha, color, group, linetype, size, weight  
e <- geom\_rug(sides = "bl") x, y, alpha, color, linetype, size  
e <- geom\_smooth(method = lm) x, y, alpha, color, fill, group, linetype, size, weight  
e <- geom\_text(aes(label = cty)) - x, y, label, alpha, angle, color, family, fontface, hjust, linetype, size, vjust

### one discrete, one continuous

f <- ggplot(mpg, aes(class, hwy))  
f <- geom\_col() x, y, alpha, color, fill, group, linetype, size  
f <- geom\_boxplot() x, y, lower, middle, upper, ymax, ymin, alpha, color, fill, group, linetype, shape, size, weight  
f <- geom\_dotplot(binaxis = "y", stackdir = "center") x, y, alpha, color, fill, group  
f <- geom\_violin(scale = "area") x, y, alpha, color, fill, group, linetype, size, weight

### both discrete

g <- ggplot(diamonds, aes(cut, color))  
g <- geom\_count() x, y, alpha, color, fill, shape, size, stroke  
g <- geom\_jitter(height = 2, width = 2) x, y, alpha, color, fill, shape, size

### THREE VARIABLES

sealsz <- with(seals, sqrt(delta\_long\*2 + delta\_lat\*2)); i <- ggplot(seals, aes(long, lat))  
i <- geom\_contour(aes(z = z)) x, y, z, alpha, color, group, linetype, size, weight  
i <- geom\_contour\_filled(aes(fill = z)) x, y, alpha, color, fill, group, linetype, size, subgroup

### continuous bivariate distribution

h <- ggplot(diamonds, aes(carat, price))  
h <- geom\_bin2d(binwidth = c(0.25, 500)) x, y, alpha, color, fill, linetype, size, weight  
h <- geom\_density\_2d() x, y, alpha, color, group, linetype, size  
h <- geom\_hex() x, y, alpha, color, fill, size

### continuous function

i <- ggplot(economics, aes(date, unemployment))  
i <- geom\_area() x, y, alpha, color, fill, linetype, size  
i <- geom\_line() x, y, alpha, color, group, linetype, size  
i <- geom\_step(direction = "hv") x, y, alpha, color, group, linetype, size

### visualizing error

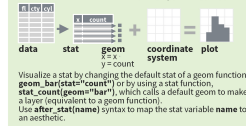
df <- data.frame(grp = c("A", "B"), fit = 4.5, se = 1.2)  
j <- ggplot(df, aes(grp, fit, ymin = fit - se, ymax = fit + se))  
j <- geom\_crossbar(fatten = 2) - x, y, ymax, ymin, alpha, color, fill, group, linetype, size  
j <- geom\_errorbar() - x, ymax, ymin, alpha, color, group, linetype, shape, size, width  
Also: geom\_errorbarh()  
j <- geom\_linerange() x, y, ymin, ymax, alpha, color, group, linetype, size  
j <- geom\_pointrange() - x, y, ymin, ymax, alpha, color, fill, group, linetype, shape, size

### maps

Draw the appropriate geometric object depending on the values of one or more discrete variables.  
map\_id\_alpha, color, fill, linetype, linewidth  
nc <- st\_read(system.file("shape/nc.shp", package = "sf"))  
ggplot(nc) +  
geom\_sf(aes(fill = AREA))

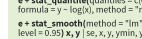
## Stats

An alternative way to build a layer.



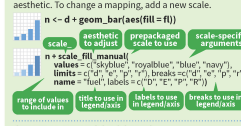
Visualize a stat by changing the default stat of a geom function, geom\_bar(stat = "count") or by using a stat function, stat\_count(geom = "bar"), which calls a default geom to make a layer (equivalent to a geom function). Use after\_stat(name) syntax to map the stat variable name to an aesthetic.

geom to use stat function geom mappings  
i <- stat\_density\_2d(aes(fill = after\_stat(level)), geom = "polygon") variable created by stat  
e <- stat\_bin(binwidth = 1, boundary = 10) x, y | count, count, density, density  
c <- stat\_count(width = 1) x, y | count, prop  
c <- stat\_density(adjust = 1, kernel = "gaussian") x, y | count, density, scaled  
e <- stat\_bin\_2d(bins = 30, drop = T) x, y, fill | count, density  
e <- stat\_bin\_hex(bins = 30) x, y, fill | count, density  
e <- stat\_density\_2d(contour = TRUE, n = 100) x, y, color, size | level  
e <- stat\_ellipse(level = 0.55, segments = 51, type = "lc")  
i <- stat\_contour(aes(z = z)) x, y, z, order | level  
i <- stat\_summary\_hex(aes(z = z), bins = 30, fun = max) x, y, z, fill | value  
i <- stat\_summary\_2d(aes(z = z), bins = 30, fun = mean) x, y, z, fill | value  
f <- stat\_boxplot(coef = 1.5) x, y | lower, middle, upper, width, ymin, ymax  
f <- stat\_ydensity(kernel = "gaussian", scale = "area") x, y | density, scaled, count, n, violinwidth, width  
e <- stat\_ecdf(n = 40) x, y | x, y  
e <- stat\_quantile(quantiles = c(0.1, 0.9), formula = y ~ log(x), method = "n") x, y | quantile  
e <- stat\_smooth(method = "lm", formula = y ~ x, se = T, level = 0.95) x, y | se, x, y, ymin, ymax  
ggplot() + xlim(-5, 5) + stat\_function(fun = dnorm, n = 20, geom = "point") x | x, y  
ggplot() + stat\_qq(aes(sample = rnorm(1000))) x, y, sample | sample, theoretical  
e <- stat\_sum() x, y, size | n, prop  
e <- stat\_summary(fun.data = "mean\_cl\_boot")  
h <- stat\_summary\_bin(fun = "mean", geom = "bar")  
e <- stat\_unique()



## Scales

Override defaults with scales package.



### GENERAL PURPOSE SCALES

Use with most aesthetics  
scale\_\*\_continuous() - Map cont' values to visual ones.  
scale\_\*\_discrete() - Map discrete values to visual ones.  
scale\_\*\_identity() - Use data values as visual ones.  
scale\_\*\_manual(values = c()) - Map discrete values to manually chosen visual ones.  
scale\_\*\_date(date, labels = "full", date, breaks = "2 weeks") - Treat date values as dates. Same as scale\_\*\_date(). See strftime for label formats.

### X & Y LOCATION SCALES

Use with x or y aesthetics (x shown here)  
scale\_x\_log10() - Plot x on log10 scale.  
scale\_x\_reverse() - Reverse the direction of the x axis.  
scale\_x\_sqrt() - Plot x on square root scale.

### COLOR AND FILL SCALES (DISCRETE)

n <- scale\_fill\_brewer(palette = "Blues")  
For palette choices:  
RColorBrewer::display.brewer.all()  
n <- scale\_fill\_grey(start = 0.2, end = 0.8, max.value = "red")  
RColorBrewer::brewer.pal()

### COLOR AND FILL SCALES (CONTINUOUS)

o <- c + geom\_dotplot(aes(fill = x))  
o <- scale\_fill\_distiller(palette = "Blues")  
o <- scale\_fill\_gradient(palette = "red", high = "yellow")  
o <- scale\_fill\_gradient2(low = "red", high = "blue", mid = "white", midpoint = 25)  
o <- scale\_fill\_gradientn(colors = topo.colors(6))  
Also: rainbow(), heat.colors(), terrain.colors(), cm.colors(), RColorBrewer::brewer.pal()

### SHAPE AND SIZE SCALES

p <- e + geom\_point(aes(shape = fl, size = cyl))  
p <- scale\_shape() + scale\_size() p <- scale\_shape\_manual(values = c(3, 7))  
p <- scale\_radius(range = c(1, 6))  
p <- scale\_size\_area(max.size = 6)

## Coordinate Systems

r <- d + geom\_bar()  
r <- coord\_cartesian(xlim = c(0, 5)) - xlim, ylim  
The default cartesian coordinate system.  
r <- coord\_fixed(ratio = 1/2) ratio, xlim, ylim - Cartesian coordinates with fixed aspect ratio between x and y units.  
r <- coord\_flip() Flip cartesian coordinates by switching x and y aesthetic mappings.  
r <- coord\_polar(theta = "x", direction = 1) theta, start, direction - Polar coordinates.  
r <- coord\_trans("sqrt") - x, y, xlim, ylim Transformed cartesian coordinates. Set xtrans and ytrans to the name of a window function.  
r <- coord\_sf(xlim, ylim, crs, Ensures all layers use a common Coordinate Reference System.

## Position Adjustments

Position adjustments determine how to arrange geoms that would otherwise occupy the same space.

s <- ggplot(mpg, aes(fl, fill = drv))  
s <- geom\_bar(position = "dodge") Arrange elements side by side.  
s <- geom\_bar(position = "fill") Stack elements on top of one another, normalize height.  
e <- geom\_point(position = "jitter") Add random noise to x and y position of each element to avoid overplotting.  
e <- geom\_label(position = "nudge") Nudge labels away from points.  
s <- geom\_bar(position = "stack") Stack elements on top of one another.  
Each position adjustment can be recast as a function with manual width and height arguments:  
s <- geom\_bar(position = position\_dodge(width = 1))

## Themes

r <- theme\_bw() White background with grid lines.  
r <- theme\_gray() Grey background (default theme).  
r <- theme\_dark() Dark for contrast.  
r <- theme\_classic() r <- theme\_light() r <- theme\_linedraw() r <- theme\_minimal() r <- theme\_void() Empty theme.  
r <- theme() Customize aspects of the theme such as axis, legend, panel, and facet properties.  
r <- labs(title = "Title") + theme(plot.title.position = "plot")  
r <- theme(panel.background = element\_rect(fill = "blue"))

## Faceting

Facets divide a plot into subplots based on the values of one or more discrete variables.

t <- ggplot(mpg, aes(cty, hwy)) + geom\_point()  
t <- facet\_grid(~ fl) Facet into columns based on fl.  
t <- facet\_grid(year ~ .) Facet into rows based on year.  
t <- facet\_grid(year ~ fl) Facet into both rows and columns.  
e <- facet\_wrap(~ fl) Wrap facets into a rectangular layout.  
Set scales to limit axis limits across facets.  
t <- facet\_grid(drv ~ fl, scales = "free") x and y axis limits adjust to individual facets:  
"free\_x" - x axis limits adjust  
"free\_y" - y axis limits adjust  
Set labeller to adjust facet label:  
t <- facet\_grid(fl ~ ., labeller = label\_both)  
t <- facet\_grid(fl ~ ., labeller = label\_bquote(alpha ~ .(fl)))

## Labels and Legends

Use labels() to label the elements of your plot.  
t <- labs(x = "New x axis label", y = "New y axis label", title = "Add a title above the plot", subtitle = "Add a subtitle below title", caption = "Add a caption below plot", alt = "Add alt text to the plot", <AES> = "New <AES> legend title")  
t <- annotate(geom = "text", x = 8, y = 9, label = "A") Places a geom with manually selected aesthetics.  
p <- guides(x = guide\_axis(dodge = 2)) Avoid crowded or overlapping labels with guide\_axis(dodge or angle).  
n <- guides(fill = "none") Set legend type for each aesthetic: color, legend, or none (no legend).  
p <- theme(legend.position = "bottom") Place legend at "bottom", "top", "left", or "right".  
n <- scale\_fill\_discrete(name = "Title", labels = c("A", "B", "C", "D", "E")) Set legend title and labels with a scale function.

## Zooming

Without clipping (preferred):  
t <- coord\_cartesian(xlim = c(0, 100), ylim = c(10, 20))  
With clipping (removes unseen data points):  
t <- xlim(0, 100) + ylim(10, 20)  
t <- scale\_x\_continuous(limits = c(0, 100)) + scale\_y\_continuous(limits = c(0, 100))



## Additional Resources

- Official ggplot2 documentation: <https://ggplot2.tidyverse.org/>
- R Graph Gallery: <https://r-graph-gallery.com/>
- Built-in datasets in R: `data()` for the full list

---

**Author:** [Mohammad Khoshroo](#)

**Date:** September 2025